

中国图书分类号 TP391;TP311

密级 公开

国际图书分类号 004

单位代码 10613

西南交通大学

硕士学位论文

面向政务领域的大模型数据查询与分析方 法研究与实现

学科专业 计算机科学与技术

学 号 2023201808

作者姓名 陈鑫海

指导教师 郑宇教授

学 院 计算机与人工智能学院

2026 年 3 月 20 日

A Thesis Submitted to
Southwest Jiaotong University for Master Degree

**Research and Implementation of Data Query and
Analysis Methods Based on Large Language Models
for the Government Affairs Domain**

Discipline Computer Science and
Technology

Student ID 2023201808

Author Chen Xinhai

Supervisor Professor Yu Zheng

School School of Computing and
Artificial Intelligence

March. 4, 2026

西南交通大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得西南交通大学或其它教育机构的学位或证书而使用过的材料。其他人员对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

本学位论文的主要创新点如下：

作者签名：_____

日期： 年 月 日

西南交通大学

学位论文使用授权

本学位论文作者完全了解西南交通大学有关保留、使用学位论文的规定，同意学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅。本人授权西南交通大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载，可以采用影印、缩印、扫描等复制手段保存、汇编本学位论文。

本学位论文属于

1. 保密□，在 年解密后适用本授权书

2. 不保密□，使用本授权书。

(请在以上方框内打“√”)

作者签名：_____ 导师签名：_____

日期： 年 月 日

摘 要

这里是中文摘要输入区。

关键词：关键词 1，关键词 2，关键词 3，关键词 4，关键词 5

Abstract

Here is for you to write the English abstract.

Keywords: keyword1, keyword2, keyword3, keyword4, keyword5

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 国内外研究现状	1
1.2.1 基于大模型的 Text-to-SQL 研究现状	1
1.2.2 基于大模型的代码生成与自动数据分析研究现状	1
1.2.3 政务数据管理与应用平台研究现状	1
1.2.4 现有研究评述	1
1.3 论文研究内容	1
1.4 论文章节安排	1
1.5 本章小结	1
第 2 章 相关理论与技术	3
2.1 大语言模型基础理论	3
2.1.1 Transformer 模型结构与自注意力机制	3
2.1.2 预训练语言模型与指令微调	3
2.1.3 上下文学习与思维链推理	3
2.1.4 大语言模型的工具调用能力	3
2.2 文本生成 SQL 技术	3
2.2.1 Text-to-SQL 任务定义与技术演进	3
2.2.2 数据库模式表示与 Schema Linking	3
2.2.3 基于预训练模型的 Text-to-SQL 方法	3
2.2.4 基于大语言模型提示的 Text-to-SQL 方法	3
2.3 文本转数据分析技术	3
2.3.1 Text-to-Analysis 的基本概念与处理流程	3
2.3.2 面向数据分析的任务规划与分解	3
2.3.3 基于代码生成与工具调用的数据分析方法	3
2.3.4 数据分析结果的解释生成与可视化表达	3
2.4 多智能体协同与上下文管理技术	3
2.4.1 多智能体系统的基本概念与协作模式	3

2.4.2 面向复杂任务的 Agent 角色划分与任务编排	3
2.4.3 多轮交互中的上下文表示与记忆机制	3
2.5 本章小结	3
第 3 章 基于逻辑增强与异构协同的政务数据查询生成方法	5
3.1 引言	5
3.2 总体框架设计	6
3.3 离线知识准备	7
3.3.1 向量库构建	7
3.3.2 政务领域知识库构建	8
3.3.3 动态样例库构建	9
3.4 基于逻辑增强的模式链接	11
3.5 异构候选生成	14
3.5.1 逻辑映射生成器	14
3.5.2 渐进分治生成器	15
3.6 基于执行反馈的查询修复	16
3.7 候选判别模型	18
3.7.1 训练样本构建	19
3.7.2 判别模型训练	19
3.7.3 配对判别与最优候选选择	20
3.8 实验与结果分析	21
3.8.1 数据集构建与实验设置	21
3.8.2 评估指标与对比基线	22
3.8.3 总体性能对比	23
3.8.4 消融实验	24
3.8.5 异构协同与候选判别分析	25
3.8.6 效率分析	27
3.8.7 公共数据集实验结果	28
3.9 本章小结	30
第 4 章 面向政务洞察的分析脚本生成方法	31
4.1 引言	31
4.2 总体框架设计	32
4.3 基于 AP-Schema 的规划驱动分析脚本生成	33
4.3.1 分析意图识别与 AP-Schema 构建	34

4.3.2 分析算子编排与可视化决策	35
4.3.3 约束化分析脚本生成	36
4.4 基于查询增强的分析执行闭环	37
4.4.1 基于查询增强的数据获取	38
4.4.2 受控执行与结果组织	39
4.5 实验与结果分析	40
4.5.1 测试任务与实验设置	40
4.5.2 结果对比与消融分析	41
4.5.3 案例分析	42
4.6 本章小结	44
第 5 章 系统设计与实现	45
5.1 引言	45
5.2 系统总体架构设计	45
5.3 业务流程设计	45
5.3.1 智能查询业务流程	45
5.3.2 智能分析业务流程	45
5.3.3 异常回退与结果追溯流程	45
5.4 系统数据存储	45
5.4.1 业务数据源接入与元数据抽取	45
5.4.2 系统数据库设计	45
5.4.3 结果、日志与缓存管理	45
5.5 多智能体协同与上下文管理实现	45
5.5.1 轻量级多智能体角色设计	45
5.5.2 查询与分析服务编排	45
5.5.3 上下文维护与状态流转	45
5.6 系统功能实现与效果展示	45
5.7 本章小结	45
结论与展望	47
致 谢	49
参考文献	51
附录 A	53
攻读硕士学位期间取得的研究成果	55

第 1 章 绪论

本章旨在阐述研究的宏观背景、理论与实践意义，并明确研究的主要内容、方法与整体结构。

1.1 研究背景与意义

1.1.1 研究背景

1.1.2 研究意义

1.2 国内外研究现状

1.2.1 基于大模型的 Text-to-SQL 研究现状

1.2.2 基于大模型的代码生成与自动数据分析研究现状

1.2.3 政务数据管理与应用平台研究现状

1.2.4 现有研究评述

1.3 论文研究内容

1.4 论文章节安排

1.5 本章小结

第 2 章 相关理论与技术

本章旨在详细介绍论文所依赖的核心理论与关键技术，为后续章节的方法设计和系统实现提供理论基础。

2.1 大语言模型基础理论

2.1.1 Transformer 模型结构与自注意力机制

2.1.2 预训练语言模型与指令微调

2.1.3 上下文学习与思维链推理

2.1.4 大语言模型的工具调用能力

2.2 文本生成 SQL 技术

2.2.1 Text-to-SQL 任务定义与技术演进

2.2.2 数据库模式表示与 Schema Linking

2.2.3 基于预训练模型的 Text-to-SQL 方法

2.2.4 基于大语言模型提示的 Text-to-SQL 方法

2.3 文本转数据分析技术

2.3.1 Text-to-Analysis 的基本概念与处理流程

2.3.2 面向数据分析的任务规划与分解

2.3.3 基于代码生成与工具调用的数据分析方法

2.3.4 数据分析结果的解释生成与可视化表达

2.4 多智能体协同与上下文管理技术

2.4.1 多智能体系统的基本概念与协作模式

2.4.2 面向复杂任务的 Agent 角色划分与任务编排

2.4.3 多轮交互中的上下文表示与记忆机制

2.5 本章小结

第3章 基于逻辑增强与异构协同的政务数据查询生成方法

3.1 引言

文本到 SQL 生成 (Text-to-SQL) 是将自然语言问题自动转换为可在关系数据库上执行的结构化查询语言的语义解析任务^[1], 是实现“智能问数”和降低数据分析门槛的关键技术。在政务领域, 随着数字政府建设的深入, 海量的人口、经济、社会等异构数据急剧增长, 使得非技术背景的公务人员能够直接、高效地从数据中获取决策支持显得尤为迫切。

本章提出的 Text-to-SQL 方法, 其任务形式化定义为: 给定用户的自然语言问题 Q 、目标数据库模式 S 、领域知识库 K 以及动态样例库 F , Text-to-SQL 任务的目标是生成一条可正确执行并返回预期结果的 SQL 查询语句, 即与用户意图最大化一致:

$$y^* \equiv \arg \max_{y \in Y} P(y \mid Q, S, K, F) \quad (3-1)$$

其中 Y 为符合 SQL 语法的候选集合, θ 为模型参数。

本章的研究目标是面向政务数据查询这一具体应用场景, 以系统性解决复杂政务语义下查询生成的正确性、执行成功率与鲁棒性问题。以一个典型的政务查询为例, 当用户提出“2023 年各区县的一般公共预算收入同比增长率是多少?”而这一看似简单的问题时, 系统需要理解“一般公共预算收入”对应的物理字段、“同比增长率”隐含的跨年度计算逻辑以及“区县”维度的分组聚合方式, 才能生成正确的 SQL 查询。这一过程涉及的业务知识远超数据库模式本身所能表达的信息边界。

与通用领域的 Text-to-SQL 任务相比, 政务数据查询面临着更为突出的复杂性挑战, 主要体现在以下方面:

(1) 业务术语的语义模糊与别名多样。政府工作存在大量专业术语及其简称、别名和口语化表达。例如, “一般公共预算收入”可能被简称为“一般预算收入”或“公共预算收入”, 而数据库中的物理字段名可能为 `ybsr` 等缩写形式。这种多对一的映射关系显著增加了模式链接的难度。

(2) 指标计算逻辑的隐含性。大量政务指标并非数据库中的直接存储字段, 而需要通过特定的计算公式推导得出。例如, “同比增长率”需要通过当期值与上期值的差值除以上期值来计算, “预算执行率”需要执行数除以预算数。这些隐含的

计算逻辑未显式存在于数据库模式中，导致大语言模型在面对此类查询时极易产生“逻辑幻觉”——即生成语法正确但业务逻辑错误的 SQL 语句。

(3) 查询结构的复杂性与多样性。SQL 查询的复杂度分布呈现明显的长尾特征：高频需求多为结构相对固定的简单查询，而低频需求则可能涉及多层嵌套子查询、跨表关联、条件聚合等复杂 SQL 结构。单一的生成范式难以同时兼顾这两类需求，对简单查询可能引入不必要的推理冗余，对复杂查询则可能因推理深度不足而导致准确率骤降。

上述问题分别对 Text-to-SQL 流程中的模式链接、候选生成和结果判别三个核心环节产生影响。基于此，本章的方法设计围绕以下三个目标展开：第一，通过构建领域知识库并将其显式注入数据库模式，同时设计一个应用于快速理解政务领域的数据库模式，以提升模式链接环节对政务语义的理解准确性；第二，通过设计异构协同的双路生成机制，平衡高频标准需求与低频复杂长尾需求的生成效率与质量；第三，通过引入执行反馈修复机制与基于微调的候选判别模型，提升最终查询结果的稳定性与鲁棒性。

3.2 总体框架设计

如图3-1所示，本章提出的基于逻辑增强与异构协同的 Text-to-SQL 方法包含离线知识准备和在线推理生成两条链路，由四个核心模块组成：离线数据预处理模块、模式链接模块、候选生成模块和候选判别模块。

离线阶段负责为在线推理提供高可用的辅助数据支撑。具体而言，该阶段从业务数据库中提取字符型列及其取值构建向量索引数据库 (*VecDB*)，为后续的值检索提供基础；通过人工梳理与结构化组织，构建包含业务术语映射、逻辑计算公式和值域约束的政务领域知识库 K ；同时，利用大语言模型对已有的“问题-SQL”对进行知识解构，生成包含中间推理链的动态样例库 F 。

在线阶段以用户的自然语言查询为输入，依次经过以下处理流程。首先，模式链接模块从原始数据库模式中检索与问题相关的表、列和值信息，并注入领域知识库中的业务术语、计算公式和值域约束，将原始数据库模式 S 转化为面向当前问题的逻辑增强模式 (LA-Schema) S^* 。随后，候选生成模块采用异构协同策略，通过逻辑映射生成器和渐进分治生成器两条并行推理路径，分别从领域逻辑映射和渐进式分解推理的角度生成多样化的 SQL 候选集。每个生成器基于大模型较高温度设置下并行采样生成候选 SQL 查询。对于生成过程中出现的语法错误或执行异常，查询修复器利用执行反馈信息引导大语言模型进行自反思修复。最终，候选判别模块通过微调的二分类选择模型对候选集进行配对比较与得分聚合，选出

最优的 SQL 查询语句作为最终输出。

各模块之间的数据流转关系形成了“知识准备—模式增强—候选生成—修复优化—判别选择”的完整闭环，其中离线知识准备为在线推理持续提供领域先验支撑，在线推理的执行结果又可反馈用于丰富样例库，实现系统能力的渐进增强。

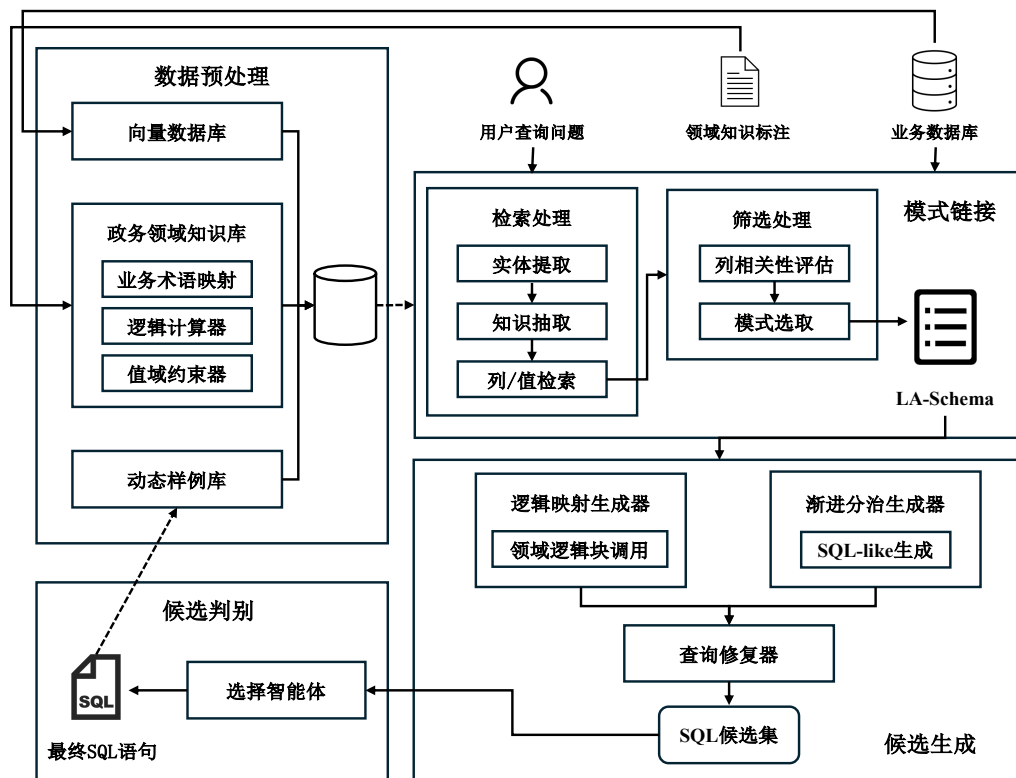


图 3-1 逻辑增强与异构协同的 Text-to-SQL 框架图

3.3 离线知识准备

离线数据预处理模块的目标是为后续在线推理生成提供低冗余、高可用的辅助数据。本模块通过构建多维度的数据知识支撑体系，为后续模式链接和候选生成过程提供精确的语义和逻辑基础。该支撑体系包含向量索引数据库、政务领域知识库以及动态样例库三个核心组件。

3.3.1 向量库构建

为保障大语言模型生成的 SQL 语句与底层数据库实际状态的一致性，本模块构建了高质量的向量索引数据库 *VecDB*，用于模式链接过程中的值检索。

具体构建流程如下。首先，从数据库中提取所有字符型列 C 及其对应的取值 v ，构造联合文本序列 $s = [C : v]$ 。通过将值的向量表示携带其所属列的上下文信

息，以帮助在检索时区分不同列下的同名值。

然后，利用预训练编码模型 BGE-m3^[2]将上述文本序列映射为高维语义向量 $\mathbf{h}$ 。此外考虑到政务数据库的规模庞大，为优化存储开销并提升检索效率，仅对字符串类型数据进行向量化。所有向量存入基于分层可导航小世界图（Hierarchical Navigable Small World, HNSW）^[3]的索引结构中。HNSW 通过构建多层跳跃链接的近邻图结构，在高维空间中实现对数级别的近似最近邻检索，相较于暴力搜索在大规模向量库上具有显著的效率优势，同时保持较高的召回率。最终向量库构建过程可表述为：

$$\begin{cases} \mathbf{h} = \text{BGEEncode}([C : v]) \\ \text{VecDB} = \{(\text{HNSW}(\mathbf{h}), C, v) \mid \forall C, v\} \end{cases} \quad (3-2)$$

向量数据库在在线阶段承担两方面的核心作用：一是通过语义相似度检索实现对用户问题中关键实体的精准召回，即使存在拼写差异或表述变化也能准确定位对应的数据库值；其次是为动态样例检索提供嵌入向量基础，使系统能够根据当前问题的语义特征检索最相关的历史样例。

3.3.2 政务领域知识库构建

针对政务数据查询中业务逻辑复杂且语义模糊的问题，本文构建了结构化的政务领域知识库 $K = \{B, L, V\}$ 。不同于传统的非结构化文本注释方式，该知识库采用结构化三元组形式存储先验业务知识，旨在为后续的模式链接和候选生成过程提供确切的逻辑增强。

知识库由以下三个组件构成：

（1）业务术语映射 B ：记录政务领域中复杂业务指标与物理数据库字段之间的对应关系。例如，三元组（“一般公共预算收入”， $ybsr$ ，“一般公共预算收入金额字段”）将业务术语映射到具体的数据库字段。

$$B = \{(t_i, c_i, d_i)\}_{i=1}^{|B|} \quad (3-3)$$

其中 t_i 为业务术语， c_i 为对应的物理字段或字段组合， d_i 为转换说明。

（2）逻辑计算器 L ：存储复杂的计算公式及聚合逻辑，将其表示为 SQL 代数表达式。例如，对于“同比增长率”这一指标，其 SQL 表达式为（当期值 - 上期值）/ 上期值 * 100。通过将复杂的业务计算逻辑预先编码为 SQL 代数表达式并以虚拟列的形式注入数据库模式 Schema，模型在生成 SQL 时可以直接调用这些预定义的逻辑块，从而规避了模型在处理大规模复杂计算公式时的推理冗余和盲

目判断。

$$L = \{(m_j, f_j, e_j)\}_{j=1}^{|L|} \quad (3-4)$$

其中 m_j 为指标名称, f_j 为计算公式的 SQL 表达式, e_j 为计算说明。

(3) 值域约束器 V : 记录特殊业务字段的取值范围或枚举值。例如, ($xzqh$, {“荣华街道”, “博兴街道”, “经济开发区”, ...}) 记录行政区划名称字段的合法取值。值域约束的引入能够帮助模型生成包含正确 WHERE 条件的 SQL 语句, 避免因值不匹配导致的查询结果为空。

$$V = \{(c_k, R_k)\}_{k=1}^{|V|} \quad (3-5)$$

其中 c_k 为字段名, R_k 为该字段的合法取值范围或枚举值集合。

知识库的来源包括政务业务术语表、字段文档、业务统计口径规则以及领域专家整理的历史问答知识。其标注过程以人机协同方式, 人工标注为主, 辅助以大语言模型辅助标注。知识的归一化组织遵循“术语—标准表达—对应字段/取值—业务说明”的统一结构, 便于在模式链接阶段进行高效检索与注入。

3.3.3 动态样例库构建

少样本 (Few-shot) 学习是辅助大语言模型进行 SQL 生成的一种重要方法。研究表明, 示例问题的质量和与当前问题的相关性对 Text-to-SQL 任务的生成效果具有关键影响^[4-5]。基于此, 本模块采用动态少样本学习策略, 而非固定的少样本集合, 以提高大语言模型在不同查询场景下的适应性和生成质量。

动态样例库的构建流程如图3-2所示。首先, 利用大语言模型对原始的“问题-SQL”对进行知识解构, 生成中间层的思维链 (Chain-of-Thought, CoT) 信息。该过程将每个样例从 $\langle \text{Query}, \text{SQL} \rangle$ 二元组扩展为包含完整推理逻辑的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组。以此构建一个逻辑表达能力更强的样例库。CoT 信息的具体组成如表3-1所示。这种结构化的推理链不仅为后续生成器提供了更丰富的推理线索, 也使得模型能够通过模仿样例中的推理过程来提升自身的逻辑表达能力。

表 3-1 思维链 (CoT) 信息的组成要素

要素	标识	说明
查询意图分析	reason	对用户问题的语义理解与意图解读
关键列	columns	查询涉及的数据库列
筛选值	values	WHERE 等筛选条件中使用的具体值
SELECT 内容	SELECT	SELECT 子句应返回的字段与表达式
类 SQL 语句	SQL-like	忽略连接条件的简化 SQL 骨架

而在在线推理阶段，当接收到用户查询 Q 时，系统启动端到端的动态样例检索流程，即从样例库 F 中选取与当前问题最相关的 k 个样例，以构建高质量的少样本提示。该流程融合了掩码问题相似性（Masked Question Similarity, MQS）^[4]与查询结构相似性的双重度量，从问法相似和 SQL 相似两个维度综合筛选样例。

具体而言，检索过程包含以下步骤。首先，对用户查询 Q 执行掩码操作，将其中的领域特定实体（如表名、列名、具体值）替换为统一占位符，得到掩码后的查询 Q' ：

$$Q' = \text{Mask}(Q) \quad (3-6)$$

随后，同样利用预训练编码模型 BGE-m3 将 Q' 映射为稠密向量表示 $v_{Q'} = \text{Enc}(Q')$ ，并在同样经过掩码处理的样例问题向量库中，使用基于 HNSW 的近似 k 近邻检索（Approximate Nearest Neighbor, ANN）算法召回一批候选样例集合 F_{cand} 。掩码操作使得相似度计算更关注问题的结构与语义特征，而非表面的实体匹配，从而有效提升跨数据库场景下的样例检索质量。

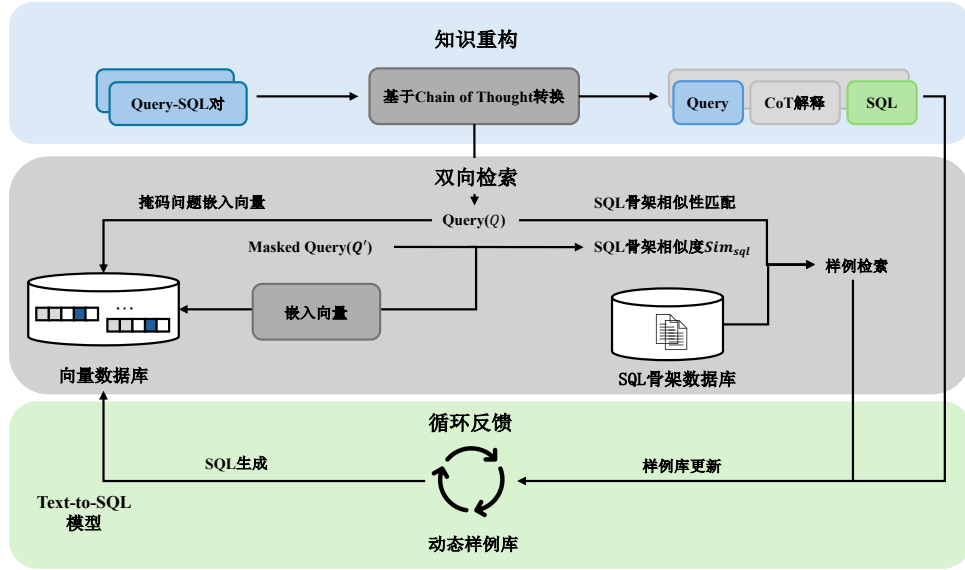


图 3-2 动态样例库构建流程图

其次在候选样例召回的基础上，系统进一步引入查询结构相似度对候选样例进行精排。具体地，先利用一个初步预测模型为 Q 生成近似 SQL 查询 $\hat{s}_Q$ 作为目标 SQL 的估计，然后将 $\hat{s}_Q$ 与每个候选样例的 SQL 查询 s_i 根据 SQL 关键词编码为二元离散语法向量，计算结构相似度 $\text{Sim}_{\text{sql}}(\hat{s}_Q, s_i)$ 。最终的样例排序综合考虑问题相似度与查询相似度，并通过预定义阈值 τ 过滤查询相似度不足的样例：

$$F_k = \text{Top-}k \left\{ \text{Sim}_{\text{ques}}(v_{Q'}, v_{Q'_i}) \mid \text{Sim}_{\text{sql}}(\hat{s}_Q, s_i) > \tau \right\}_{f_i \in F_{\text{cand}}} \quad (3-7)$$

其中 Sim_{ques} 为掩码问题向量间的相似度, F_k 即为最终选取的 k 个动态样例。选定的样例以 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组的形式拼接至提示模板中, 与数据库模式、检索到的相似值及领域规则共同构成完整的少样本提示, 以引导大语言模型生成目标 SQL。

最终的动态样例库具备渐进更新的能力。当渐进分治生成器产生的候选 SQL 被判别模型确认为最优答案后, 该查询及其对应的推理过程可作为新的样例加入样例库 (可控制是否开启), 实现样例库的持续丰富与质量提升。

3.4 基于逻辑增强的模式链接

模式链接是 Text-to-SQL 任务中的关键环节, 其目标是将用户的自然语言查询与数据库模式中的元素 (包括表、列和值) 相关联, 从庞大的数据库模式中筛选出最小必要且包含丰富语义信息的数据库子模式^[6]。本节提出一种基于逻辑增强的模式链接方法, 通过动态检索元素信息、选取关键模式片段并注入领域逻辑信息, 将原始数据库模式 S 转化为面向特定问题 Q 的最小增强模式 S^* 。

该方法的处理流程包含检索、筛选和增强三个阶段, 其完整算法描述如算法3.1所示。

检索阶段旨在从数据库中搜索与查询潜在关联的值和列。该阶段首先以用户查询 Q 、动态样例 F_u 及领域知识库 K 为输入, 利用大语言模型识别用户问题中的关键词与实体, 获取实体集合 W_{entity} 。随后, 对 W_{entity} 中的每个实体 w 分别执行列检索与值检索。列检索基于实体 w 与列描述之间的语义相似度, 筛选每个实体对应的 Top- k 候选列。值检索基于 HNSW 索引从向量数据库中快速召回语义最近邻候选集 V_{cand} , 并利用嵌入向量的余弦相似度进行精排, 以实现目标值及其所属列的精准定位。检索到的候选列、匹配值及其所属表共同构成候选模式片段集合 S_{cand} 。

筛选阶段旨在将检索得到的表、列、值集合缩减为生成 SQL 所需的最小数据库模式。此过程根据检索得到的候选列元素集合, 让大语言模型评估每个列与用户查询的相关性, 过滤无关列, 最终得到最小数据库模式 S_{filter} 。该筛选步骤能够有效降低后续生成阶段的输入规模, 减少因冗余信息导致的模型注意力分散和幻觉问题。

增强阶段是本方法的核心创新所在。在获得最小数据库模式后, 系统从领域知识库 K 中提取与当前查询相关的业务知识, 并将其显式注入模式表示中, 形成逻辑增强模式 LA-Schema。LA-Schema 的组织方式参照 M-Schema^[7]的半结构化格式, 以层次化的方式展示数据库、表和列之间的结构关系, 并在此基础上添加政务

算法 3.1: 基于逻辑增强的模式链接算法

Input: 用户查询 Q , 原始数据库模式 S , 列检索召回数 k , 动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$, 领域知识库 $K = \{B, L, V\}$, 向量数据库 VecDB, 大语言模型 LLM

Output: 最小增强模式 S^*

```

1 初始化: 候选模式片段集合  $S_{\text{cand}} \leftarrow \emptyset$ ;
   // 阶段一: 检索
2   $W_{\text{entity}} \leftarrow \text{LLM}(Q, F_u, K)$ ; // 实体提取
3  foreach  $w \in W_{\text{entity}}$  do
4       $C_{\text{topk}} \leftarrow \text{SemanticSearch}(w, C, k)$ ; // 列检索
5       $V_{\text{cand}} \leftarrow \text{HNSW}(w, \text{VecDB})$ ; // 值近邻召回
6       $\text{val} \leftarrow \arg \max_{v \in V_{\text{cand}}} \text{Sim}(\text{Enc}(w), \text{Enc}(v))$ ; // 语义精排
7       $S_{\text{cand}} \leftarrow S_{\text{cand}} \cup \{C_{\text{topk}}, \text{val}, \text{Table}(\text{val})\}$ ;
8  end
   // 阶段二: 筛选
9   $S_{\text{filter}} \leftarrow \text{LLM}(Q, S_{\text{cand}})$ ; // 列相关性评估与筛选
   // 阶段三: 增强
10  $C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}} \leftarrow \text{GetKnowledge}(W_{\text{entity}}, B, L, V)$ ;
11  $S^* \leftarrow \text{TransLASchema}(S_{\text{filter}}, C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}})$ ; // 注入知识并转换为 LA-Schema
12 return  $S^*$ 

```

领域业务知识的显式表达。具体的增强内容包括三个层面:

(1) **术语归一与别名扩展:** 对于知识库 B 中与当前查询匹配的业务术语映射, 在对应字段上添加 **Business Term** 标记, 标注该字段在业务语义上的对应关系。例如, 当字段 `ybsr` 被识别为“一般公共预算收入”的物理存储字段时, LA-Schema 中会以 `[Business Term: 一般公共预算收入  $\rightarrow$  ybsr]` 的形式进行标注。

(2) **虚拟逻辑列注入:** 对于知识库 L 中匹配的计算指标, 在 LA-Schema 中创建虚拟列 (Virtual columns), 并附带预定义的 SQL 计算公式。例如, 对于“同比增长率”指标, 系统会注入一个标记为 `[Logical Term: 同比增长率, Formula: (当期值 - 上期值) / 上期值 * 100]` 的虚拟字段。这样, 大语言模型在生成 SQL 时可以直接引用预定义的计算逻辑, 而无需从零开始推理复杂的业务公式。

(3) **列值提示与约束:** 对于知识库 V 中匹配的值域约束以及通过值检索获得的匹配值, 在对应列上添加 **Value Term** 标记, 提示模型在 WHERE 条件中使用正确的匹配值。

为直观说明 LA-Schema 的增强效果, 以下给出一个对比示例。对于查询“帮我找出所有状态为已完工的民生项目, 并计算它们的资金结余率。”, 原始 DDL Schema、M-Schema 与 LA-Schema 的展示结果如表3-2所示。

表 3-2 原始 DDL Schema、M-Schema 与 LA-Schema 对比示例

模式类型	表示内容
DDL Schema	<pre>CREATE TABLE t_gov_proj(p_id INTEGER PRIMARY KEY, p_name VARCHAR, amt_tot INTEGER, amt_act INTEGER, p_type TINYINT, status TINYINT, p_year INTEGER)</pre>
M-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [(p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), (status: TINYINT, 状态, Examples:[0,1]), (p_year: INTEGER, 年份, Examples:[2023,2024,2025])]</pre>
LA-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [Physical Columns: (p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), Business Term:{" 民生项目": " 民生工程"}, Value Term:{" 民生工程":1," 基础设施":2," 产业扶持":3}), (status: TINYINT, 状态, Examples:[0,1]), Value Term:{" 进行中":0," 已完工":1}), (p_year: INTEGER, 年份, Examples:[2023,2024,2025]) Virtual Columns: (balance_rate: Logical Term, 资金结余率, Examples:[0.1,0.2,0.3], Formula:"(amt_tot-amt_act)/amt_tot")]</pre>

可以看到，原始 DDL Schema 仅提供了最基本的列名与数据类型，缺乏任何语

义信息；M-Schema 在此基础上补充了中文列描述和示例值，但仍无法表达业务术语映射、值域约束等领域知识。而 LA-Schema 通过显式注入 Business Term、Value Term 与 Virtual Columns 等增强信息，使大语言模型能够准确理解“民生项目”对应的物理字段取值、“已完工”的编码映射以及“资金结余率”的计算公式，从而显著降低模式链接阶段的错误率。

3.5 异构候选生成

在政务数据查询场景中，用户查询的复杂度分布呈现显著的长尾特征^[8]，即高频需求往往是结构相对固定的标准查询，而低频需求可能涉及多层嵌套、多表关联和复杂聚合等高难度 SQL 结构。单一的生成路径在处理这种高度异构的查询分布时存在固有局限：对于简单查询，过度复杂的推理策略会引入冗余步骤并增加出错风险；对于复杂查询，直接映射式的生成方式又难以捕获深层的逻辑关系^[6]。

为此，本文设计了一种异构协同生成框架，包含旨在捕获高频业务逻辑的逻辑映射生成器以及处理复杂推理深度的渐进分治生成器。通过并行驱动两个推理范式截然不同的生成器，构建多样化的 SQL 候选池，以提升系统对各种复杂度查询的覆盖能力。生成阶段允许大语言模型在非零温度^[9]设置下执行随机采样，每个生成器并行生成 n_c 个候选 SQL 查询，最终总计产生 $2n_c$ 个多样性的 SQL 候选集。

3.5.1 逻辑映射生成器

逻辑映射生成器深度集成了本章提出的 LA-Schema 架构，其推理机制建立在领域逻辑映射的基础之上。该生成器通过引导大语言模型对 LA-Schema 中的领域先验进行识别与调用，其包括业务术语的语义对齐、虚拟逻辑列的公式引用以及值域约束的条件匹配。以有效地将复杂的 SQL 构造过程从底层的算子推导转化为对预设逻辑块的检索与组装。其完整流程如算法3.2所示。

算法中，BuildRules() 构建的映射规则指令明确要求模型遵循以下约束：（1）优先使用 LA-Schema 中标记为 Virtual Column 的虚拟逻辑列，直接在 SQL 中调用其预定义的计算公式；（2）识别并转换 Business Term 标记的业务术语，使用映射后的物理字段替换原始表达；（3）利用 Value Term 标记的匹配列值，确保 WHERE 条件中使用与数据库一致的标准取值；（4）直接输出最终的可执行 SQL，不作过多中间解释。生成阶段在非零温度下对同一提示执行 n_c 次独立采样，以获得多样化的候选 SQL 集合 $\mathcal{C}_{lm}$ 。

该生成器的核心优势在于效率高、对常见查询模式适应性强。对于政务场景

算法 3.2: 逻辑映射生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 $\mathcal{C}_{lm}$

```

1 初始化: 候选集合  $\mathcal{C}_{lm} \leftarrow \emptyset$ , 提示  $P \leftarrow \emptyset$ ;
   // 构建提示模板
2  $P \leftarrow P \oplus \text{FormatFewShot}(F_u)$ ;           // 拼接动态少样本示例
3  $P \leftarrow P \oplus S^*$ ;                         // 拼接 LA-Schema 增强模式
4  $R \leftarrow \text{BuildRules}()$ ;                     // 构建映射规则指令
5  $P \leftarrow P \oplus R \oplus Q$ ;                   // 拼接规则指令与用户查询
   // 多次采样生成候选 SQL
6 for  $i \leftarrow 1$  to  $n_c$  do
7    $\hat{y}_i \leftarrow \text{LLM}(P, \text{temperature} > 0)$ ; // 随机采样生成 SQL
8    $\mathcal{C}_{lm} \leftarrow \mathcal{C}_{lm} \cup \{\hat{y}_i\}$ ;
9 end
10 return  $\mathcal{C}_{lm}$ 

```

中高频出现的指标查询、条件筛选和简单聚合等标准需求, 逻辑映射生成器能够充分利用 LA-Schema 中预置的领域逻辑, 快速而准确地完成 SQL 生成, 避免了不必要的推理链条展开。

3.5.2 渐进分治生成器

与逻辑映射生成器的直接映射策略不同, 渐进分治生成器采用渐进式生成与动态少样本的联合方式, 以应对知识库未覆盖的复杂需求, 如多层嵌套逻辑、多条件组合聚合以及跨表关联查询等。

渐进式生成策略本质上是一种面向 Text-to-SQL 任务定制化的思维链方法^[10], 其核心在于将复杂的 SQL 生成任务拆解为多维度、结构化的子任务序列, 引导大语言模型通过逐步推理深度理解查询逻辑。该策略借鉴了 CHASE-SQL^[6]中的分治思想和 OpenSearch-SQL^[5]中的 SQL-Like 中间表示设计, 但针对政务场景做了两方面适配: 一是在推理链条中保留了 LA-Schema 的逻辑增强信息, 使分步推理能够参考领域知识; 二是引入了类 SQL 中间表示, 允许模型先构建查询骨架再填充语法细节, 降低了一次性生成完整 SQL 的复杂度。其完整流程如算法3.3所示。

算法中, 渐进推理指令 BuildCoTInstr() 要求模型在单次生成中按照表3-1所定义的结构化格式, 以自回归方式依次输出意图分析 (reason)、涉及列 (columns)、筛选值 (values)、SELECT 内容以及 SQL-like 骨架共五个部分, 前序输出自然成为后续生成的上下文, 形成递进式的推理链。随后, 系统将生成的 SQL-like 骨架作为上下文, 再次调用大语言模型补充完整的 JOIN 条件和语法细节, 生成最终可

算法 3.3: 渐进分治生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 $\mathcal{C}_{pd}$

- 1 初始化: 候选集合 $\mathcal{C}_{pd} \leftarrow \emptyset$, 提示 $P \leftarrow \emptyset$;
// 构建提示模板
- 2 $P \leftarrow P \oplus \text{FormatCoTShot}(F_u)$; // 拼接 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 格式示例
- 3 $P \leftarrow P \oplus S^*$; // 拼接 LA-Schema 增强模式
- 4 $P \leftarrow P \oplus \text{BuildCoTInstr}() \oplus Q$; // 拼接渐进推理指令与用户查询
// 多次采样生成候选 SQL
- 5 **for** $i \leftarrow 1$ **to** n_c **do**
 | // 意图分析 $\rightarrow$ 涉及列与值 $\rightarrow \text{SELECT} \rightarrow \text{SQL-like}$
 6 | $(\text{reason}_i, \text{cols}_i, \text{vals}_i, \text{sel}_i, \hat{y}'_i) \leftarrow \text{LLM}(P, \text{temperature} > 0)$;
 7 | $\hat{y}_i \leftarrow \text{LLM}(P, \hat{y}'_i)$; // 基于 SQL-like 生成最终 SQL
 8 | $\mathcal{C}_{pd} \leftarrow \mathcal{C}_{pd} \cup \{\hat{y}_i\}$;
 9 **end**
- 10 **return** $\mathcal{C}_{pd}$

执行的 SQL 查询。动态少样本示例以完整的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组呈现, 为模型提供渐进推理的范式参考。生成阶段同样在非零温度下执行 n_c 次独立采样, 产生多样化的候选集合 $\mathcal{C}_{pd}$ 。

该生成器的核心优势在于对复杂查询的处理能力。当查询涉及多层嵌套逻辑、复杂的条件组合或跨表关联时, 分步推理机制能够引导模型逐步构建查询结构, 避免因一步到位生成长 SQL 而导致的逻辑遗漏或结构错误。同时, 渐进生成器输出的结构化推理过程本身也具有可解释性, 有助于后续的错误分析和结果追溯。

此外, 渐进分治生成器与动态样例库之间存在正向反馈关系。当该生成器产生的候选 SQL 经判别后被确认为正确答案时, 其完整的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 输出可作为新的高质量样例纳入动态样例库, 为后续相似问题的处理提供更精准的参考。

3.6 基于执行反馈的查询修复

无论是逻辑映射生成器还是渐进分治生成器, 其产生的 SQL 候选集在某些情况下不可避免地存在逻辑或语法错误^[6,11], 这些错误会导致查询无法正确执行或返回空结果。为提升候选集的整体质量, 本文引入了一个基于执行反馈的查询修复模块, 作为候选生成后、判别选择前的鲁棒性增强环节。

在实际政务查询场景中, 常见的 SQL 错误类型包括: (1) 字段名不存在或拼写错误, 通常由模式链接阶段的遗漏或大语言模型的幻觉引起; (2) 表连接关系错误, 如遗漏必要的 JOIN 条件或使用了错误的连接键; (3) SQL 语法错误, 如括

号不匹配、关键字使用不当等；（4）聚合逻辑错误，如 GROUP BY 子句缺失必要字段或聚合函数使用不当。

查询修复器的工作机制基于自反思（Self-Reflection）方法^[12]。整个过程如图3-3所示。对于候选集中的每条 SQL 查询，系统首先在目标数据库上执行该查询，捕获执行结果或错误信息。若执行成功则保留该候选；若执行失败（如语法错误），则将执行器的反馈信息包括具体的错误描述以及原始用户问题、LA-Schema 信息一并构造为修复提示，送入大语言模型进行反思与修复。修复后的 SQL 会再次执行以验证其有效性。

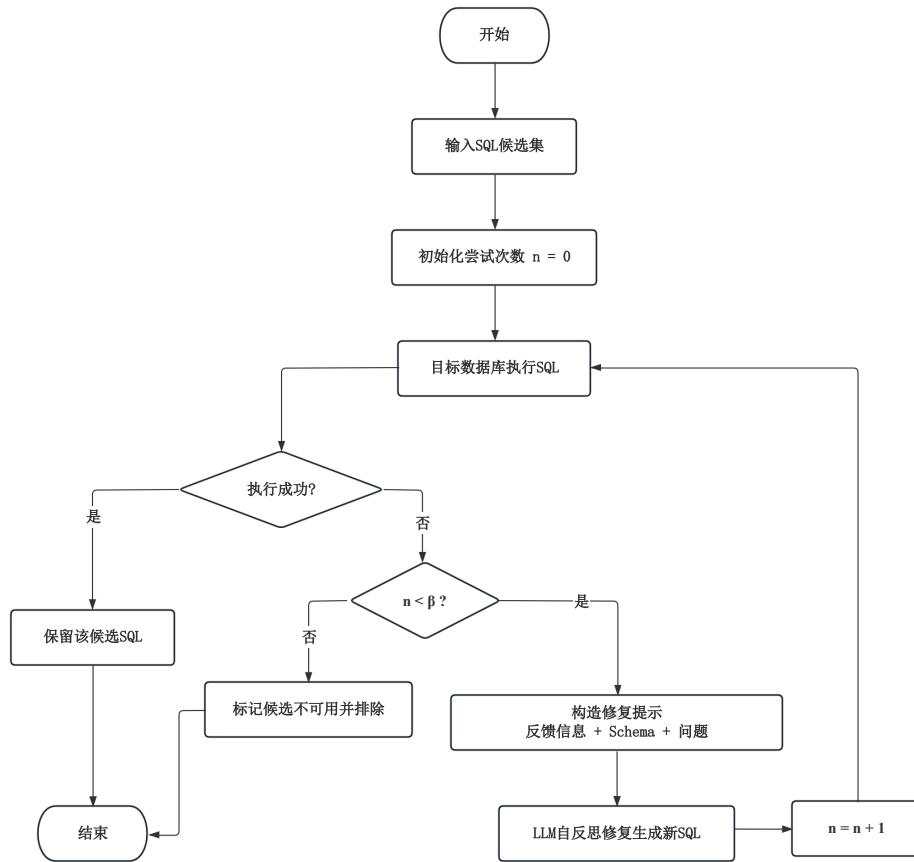


图 3-3 查询修复流程图

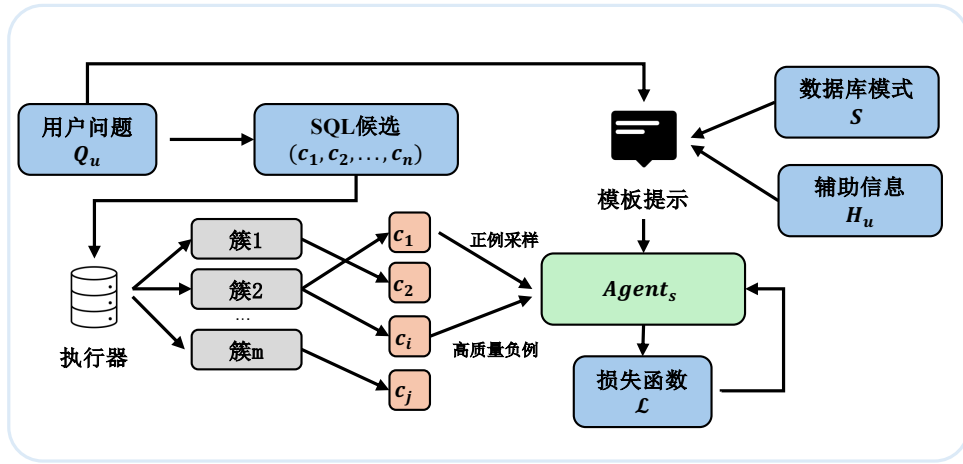
为避免无限迭代带来的效率损耗和不收敛风险，修复过程设置了最大尝试次数 β （本文设定 $\beta = 3$ ）。当满足以下任一条件时，修复过程终止：（1）修复后的 SQL 执行成功；（2）达到最大尝试次数 β 。若修复在 β 次尝试后仍未成功，则保留最后一次修复的结果或标记该候选为不可用，在后续判别阶段予以排除。

查询修复器的引入有效降低了候选集中的无效查询比例，为后续的候选判别模块提供了更高质量的输入，从而间接提升了最终查询结果的准确率。

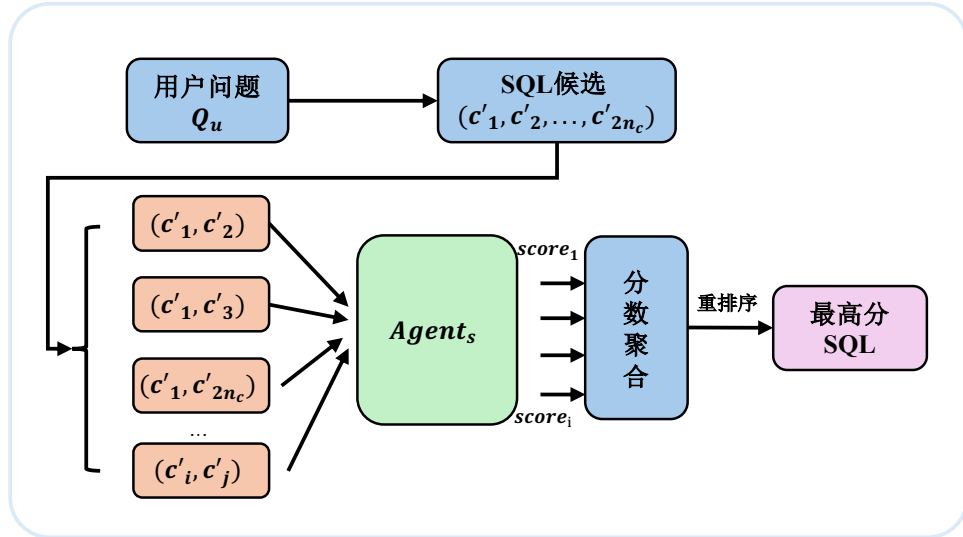
3.7 候选判别模型

经过异构候选生成与查询修复后，系统可为每个用户问题构建包含 $2n_c$ 个候选 SQL 的查询池。本节的目标是从该候选集中可靠地选出最终正确的 SQL 查询。

现有许多 Text-to-SQL 方法采用候选查询的一致性投票（Self-Consistency）策略，以得票最多的结果作为最终输出^[5]。然而，在复杂业务查询场景中，最一致的答案并不总是正确答案^[6]，其一致性选择策略与上界精度存在一定的性能差距。为此，本文构建选择智能体 $Agent_s$ ，并将候选选择建模为配对偏好判别问题。具体而言， $Agent_s$ 以用户问题、辅助提示信息、相关增强模式以及一对候选 SQL 为输入，判断两者中哪一个更可能正确，再通过两两比较与得分聚合确定最终输出。整体流程如图3-4所示。



(a) 离线训练



(b) 在线推理

图 3-4 候选判别整体流程

3.7.1 训练样本构建

判别模型的训练数据来源于在训练集上运行候选生成模块所得到的候选 SQL 集合。对于训练集中的每个问题 Q_u ，系统首先利用两个异构生成器产生 $2n_c$ 个候选 SQL，并在目标数据库上执行所有候选查询。设标准 SQL 的执行结果为 R_u^* ，候选 c_k 的执行结果为 R_k 。系统依据执行结果对候选进行聚类，将执行结果相同的候选划分至同一簇，再将各簇对应的执行结果与 R_u^* 进行比较：若二者一致，则该簇记为正确簇；否则记为错误簇。

在同时存在正确簇与错误簇的情况下，系统从两类簇中抽取候选，构造配对训练样本：

$$T_{ij} = (Q_u, H_u, c_i, c_j, S_{ij}^*, y_{ij}), \quad (3-8)$$

其中， Q_u 表示用户问题， H_u 表示辅助提示信息， c_i 与 c_j 表示两个待比较的候选 SQL， S_{ij}^* 表示 c_i 和 c_j 所涉及增强数据库模式的并集， $y_{ij} \in \{0, 1\}$ 为监督标签。当 c_i 来自正确簇且 c_j 来自错误簇时，记 $y_{ij} = 1$ ；反之记 $y_{ij} = 0$ 。

在样本构建过程中重点挖掘高价值负例。所谓高价值负例，是指执行结果与正确答案接近、但查询逻辑存在关键偏差的错误 SQL，例如筛选条件边界不同、聚合范围不一致或连接关系错误等。相较于随机负例，这类样本更能反映真实业务场景中的判别难点。对于训练集中未能生成正确候选的问题，本文仅在离线样本构造阶段引入标准 SQL 的执行结果，用于引导生成器产生更多与正确答案相似但不完全相同的错误候选，从而提升高价值负例的比例；该信息不作为判别模型在线推理时的输入，以避免信息泄漏。

3.7.2 判别模型训练

系统以 Qwen-8B 为基础模型构建选择智能体 Agent_s ，并采用指令式监督微调方式^[13](Instruction Supervised Fine-Tuning, ISFT) 进行训练。对于每个训练样本 T_{ij} ，系统将用户问题 Q_u 、辅助提示信息 H_u 、模式并集 S_{ij}^* 以及候选 SQL(c_i, c_j) 按统一模板组织为输入序列，要求模型输出二元判别结果，以表示两者中哪一个更可能为正确查询。由此， Agent_s 学习的是如下条件偏好概率：

$$p_{\theta}(y_{ij} = 1 \mid Q_u, H_u, S_{ij}^*, c_i, c_j), \quad (3-9)$$

其中 θ 表示大模型经微调后的模型参数。

训练目标采用二分类交叉熵损失函数：

$$\mathcal{L}_{\text{pair}} = -y_{ij} \log p_{\theta}(y_{ij} = 1 \mid T_{ij}) - (1 - y_{ij}) \log p_{\theta}(y_{ij} = 0 \mid T_{ij}). \quad (3-10)$$

考虑到高价值负例对模型判别能力的提升更为显著, 本文进一步引入样本权重 w_{ij} , 得到总体优化目标:

$$\mathcal{L} = \sum_{(i,j)} w_{ij} \mathcal{L}_{\text{pair}}. \quad (3-11)$$

其中, 对语义接近但逻辑错误的高价值负例赋予更高权重, 以增强模型对细微差异的敏感性。为兼顾训练效率与模型性能, 本文采用参数高效微调策略对大模型进行训练, 使其学习候选 SQL 之间的相对优劣关系且不丢失基础模型理解能力。

3.7.3 配对判别与最优候选选择

在线推理阶段, 系统基于配对比较与得分聚合机制, 从候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$ 中选出最终 SQL, 其完整流程如算法3.4所示。

算法 3.4: SQL 候选集选取算法

Input: 候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$, 用户问题 Q_u , 目标数据库 DB , 增强数据库模式 S^* , 选择智能体 Agent_s

Output: 最佳 SQL 查询 c_f

```

1 foreach  $c_i \in C$  do
2   |  $\text{score}_i \leftarrow 0$ ;
3 end
4 foreach  $(c_i, c_j) \in C \times C$  且  $i \neq j$  do
5   | if  $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$  then
6   |   |  $\text{winner} \leftarrow i$ ; // 结果一致时, 无需调用判别模型
7   | else
8   |   |  $S_{ij}^* \leftarrow \text{SchemaUnion}(c_i, c_j, S^*)$ ;
9   |   |  $\text{winner} \leftarrow \text{Agent}_s(Q_u, S_{ij}^*, c_i, c_j)$ ;
10  | end
11  |  $\text{score}_{\text{winner}} \leftarrow \text{score}_{\text{winner}} + 1$ ;
12 end
13  $c_f \leftarrow \arg \max_{c_i \in C} \text{score}_i$ ;
14 return  $c_f$ 

```

对于任意两个不同候选 (c_i, c_j) , 系统首先在目标数据库 DB 上执行二者并比较执行结果。若 $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$, 则说明二者在功能上等价, 无需调用判别模型; 若二者执行结果不同, 则构造其相关增强模式并集 $S_{ij}^* = \text{SchemaUnion}(c_i, c_j, S^*)$, 并调用 Agent_s 进行二元判别。考虑到输入顺序可能影响模型输出, 系统对每一对

候选同时比较 (c_i, c_j) 与 (c_j, c_i) 两个方向，并将胜者分别累加得分，从而减轻顺序偏差。最终，系统选择累计得分最高的候选作为输出：

$$c_f = \arg \max_{c_i \in C} \text{score}_i. \quad (3-12)$$

若出现平局，系统优先选择执行成功且返回非空结果的候选；若仍无法区分，则进一步选择 SQL 语句长度较短的候选作为最终结果。通过上述机制，本文将候选选择由一致性投票转化为基于配对偏好的全局竞争过程，从而更有效地利用候选间的细粒度差异信息，提高最终 SQL 的选取准确率。

3.8 实验与结果分析

本章首先介绍实验设置，包含数据集构建与选用、评估指标定义、对比模型。然后针对政务领域数据集上进行对比实验、消融实验、异构协同与判别分析及效率分析。最后本文还在公共数据集 BIRD^[14]、Spider^[15] 上进行实验，以评估本文方法在泛用数据集上的适用性与稳健型。

3.8.1 数据集构建与实验设置

为全面评估本文方法在政务领域的有效性，实验采用自建政务数据集进行评估。鉴于政务数据的隐私敏感性，本文基于“多源采集—逻辑注入—混合标注”的流程构建了面向政务场景的 Text-to-SQL 评测数据集。

数据集的构建流程分为三个阶段：

(1) **多源数据采集与脱敏**。数据来源于某市经济开发区政务平台，原始数据涵盖财政预算、公共资源和政务项目三个领域的 5 个数据库，共计 33 张数据表。为保障数据安全，对涉及个人隐私的敏感信息（如身份证号、联系方式等）进行了加盐哈希的严格脱敏处理，确保数据在技术上不可逆向还原。

(2) **基于领域知识库的逻辑注入**。通过人工梳理 20 余个核心业务指标，将包括预算执行率、同比增长率、集中采购率等在内的隐性业务逻辑显性化地录入标注元数据中。确保数据集包含足够多的需要领域知识。

(3) **人机协同混合标注**。问答对的生成采用人机协同的方式：首先利用 Qwen 大语言模型^[16]根据原始数据库模式生成种子问题和初始 SQL，然后由具备政务业务背景的标注人员对生成结果进行核验和纠正，确保问题的自然性和 SQL 的正确性。

最终构建的数据集包含 480 条高质量的自然语言问题—SQL 查询对，按照 7:3 的比例划分为训练集和测试集。训练集用于候选判别模型微调，而在训练样本构

建阶段产生共计近 5000 个元组。测试集用于最终端到端性能评估。数据集中约 38.5% 的问题涉及复杂查询（包括多表连接、嵌套子查询、复杂聚合等）。数据集统计信息如表3-3所示。

表 3-3 政务数据集 GovSQL 统计信息

划分	领域数	数据库数	数据表数	字段数	问题-SQL 对数	复杂查询占比/%
训练集	3	5	33	217	336	37.8
测试集	3	5	33	217	144	40.3
合计	3	5	33	217	480	38.5

在实验配置方面, 本文使用 Qwen3-Coder-30B-A3B-Instruct^[16]作为基础调用大语言模型, 用于模式链接、候选生成和查询修复等主要环节; 使用 Qwen3-8B^[16]作为候选判别模型的基础模型进行微调训练。

存储引擎使用 Milvus 作为向量库构建存储, PostgreSQL 作为领域知识库, 并联合使用 Milvus 和 PostgreSQL 实现动态样例库。在工程上使用 Redis 作为热点缓存工具, 提高调用性能。联合使用 Milvus、PostgreSQL 作为向量库、知识库以及样例库的存储引擎。

实现设置上, 动态样例检索的样本数 n_f 设为 5, 此外样例库实验评估阶段关闭在线增量更新, 避免测试数据泄漏。模式链接阶段, 候选列的召回数 k 设为 10。除候选生成外的其他 LLM 调用阶段温度参数 $temperature = 0.3$ 。而在候选生成阶段, 每个生成器在温度参数 $temperature = 0.7$ 的设置下各采样生成 $n_c = 10$ 个候选 SQL。

3.8.2 评估指标与对比基线

本文采用执行准确率 (Execution Accuracy, EX) 作为主要评估指标。EX 通过比较预测 SQL 查询与参考 SQL 查询在目标数据库上的执行结果来判定正确性:

$$EX = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{ans}(SQL_i^{\text{pred}}, DB) = \text{ans}(SQL_i^{\text{gold}}, DB)] \quad (3-13)$$

其中 N 为测试集样本数, $\text{ans}(SQL, DB)$ 表示 SQL 在数据库 DB 上的执行结果。EX 指标关注的是查询的功能等价性而非形式一致性, 即允许 SQL 在写法上存在差异, 只要执行结果相同即视为正确。

此外, 本文在异构协同与候选判别分析和效率分析中还使用了以下辅助指标: 判别模型的二分类准确率以及端到端平均响应时延。

对比基线分为以下三类：

(1) **开源基础大模型**。该类基线采用零样本（zero-shot）Text-to-SQL 提示直接生成 SQL，不引入任何额外的优化策略，旨在衡量大语言模型本身的 SQL 生成能力下限。具体包括：DeepSeek-V3^[17]，本地部署，采用零样本提示生成 SQL；Qwen3-Coder-30B-A3B-Instruct^[16]，本地部署，同样采用零样本提示生成 SQL。

(2) **基于提示词的无微调方法**。该方法通过精心设计的提示工程策略提升 SQL 生成质量，无需对模型参数进行微调。具体包括：DAIL-SQL^[4]，通过为不同问题检索语义相似的问题-SQL 对作为少样本示例，引导大语言模型生成 SQL；OpenSearch-SQL^[5]，通过动态上下文学习与一致性对齐策略，以多智能体协同的方式生成 SQL；Alpha-SQL^[18]，基于蒙特卡洛树搜索建模 Text-to-SQL 生成过程，配合动态生成推理动作与自监督奖励函数进行搜索式 SQL 生成。

(3) **基于监督微调的方法**。该方法通过在 Text-to-SQL 数据上对模型进行监督微调，使模型习得面向 SQL 生成的专用能力。具体包括：XiYan-SQL^[7]，融合监督微调与上下文学习，并使用 M-Schema 增强模式理解，以多生成器集成的方式生成 SQL；CHASE-SQL^[6]，通过多路径候选生成器产生多样化的 SQL 候选，并使用经偏好优化微调的选择代理进行候选判别；OmniSQL^[19]，通过自动化合成百万级高质量 Text-to-SQL 数据及思维链标注，基于开源大模型进行大规模监督微调。

为保证对比的公平性，上述基线方法（除开源基础大模型和 OmniSQL 外）均使用与本文相同的 Qwen3-Coder-30B-A3B-Instruct 作为基础 LLM。

3.8.3 总体性能对比

表3-4展示了各方法在政务数据集上的执行准确率对比结果。本文方法取得了 91.47% 的执行准确率，显著优于所有对比基线。

表 3-4 不同 Text-to-SQL 方法在政务数据集上的性能对比

类别	方法	执行准确率/%
开源基础大模型	DeepSeek-V3	64.90
	Qwen3-Coder-30B-A3B-Instruct	69.17
基于提示词无微调	DAIL-SQL	74.42
	OpenSearch-SQL	76.51
	Alpha-SQL	80.27
基于监督微调	OmniSQL	71.56
	CHASE-SQL	81.33
	XiYan-SQL	<u>83.83</u>
本文方法	Ours	91.47

从结果中可以得出以下分析：

以 DeepSeek-V3 和 Qwen3-Coder 为代表的零样本方法,其准确率普遍处于 65% 至 70% 之间。分析发现,这类方法虽然具备一定的 SQL 语法生成能力,但在面对政务场景中晦涩的物理列名和复杂的业务指标时,极易产生“语义幻觉”,即模型臆造了不存在的列名或使用了错误的计算逻辑,无法准确映射底层的业务规则。

DAIL-SQL 和 Alpha-SQL 等基于提示词工程的方法通过优化上下文样本选择在一定程度上提升了性能,其中 Alpha-SQL 达到了 80.27%。然而,这类方法本质上仍属于隐式推理范式,模型必须在有限的上下文窗口内同时完成语义理解和逻辑推演。在处理高嵌套、跨表计算等复杂政务查询时,上下文中的有效信息容易被稀释,推理链条断裂的概率显著增加。

XiYan-SQL 和 CHASE-SQL 等包含微调组件的方法表现相对优异,分别达到了 83.83% 和 81.33%。这表明监督微调在特定领域数据集上确实能带来增益效果。但即便经过微调,如果模型不具备实时的领域逻辑增强支撑,在面对政务业务中涉及复杂计算公式和动态变化的业务规则时,仍然难以突破 85% 的准确率瓶颈。

相比之下,本文方法相较于最强基线 XiYan-SQL 实现了 7.64 个百分点的提升,相较于 CHASE-SQL 实现了 10.14 个百分点的提升。这一显著改善主要归因于三方面的协同作用: LA-Schema 的逻辑增强机制有效缓解了政务领域的语义幻觉问题;异构协同生成策略扩大了候选池的多样性和覆盖度;微调判别模型则从多样化的候选中精准筛选出最优答案。

3.8.4 消融实验

为深入探究系统中各核心模块的有效性,本文设计了消融实验。通过在完整模型 (Full Model) 的基础上分别移除或替换关键模块,观察其在政务数据集上的执行准确率变化。实验结果如表3-5所示,移除任何一个模块均会导致性能下降,证明了各模块的不可或缺性。

w/o HNSW。在数据预处理阶段,若不使用 HNSW 索引进行向量值检索,而直接采用语义匹配的单阶段值检索,执行准确率降至 89.32%。这证明了 HNSW 索引在处理大规模政务元数据时,能够更高效且精准地召回与查询相关的数据库值。

w/o Dynamic Few-shot。移除动态样例库后,模型仅依赖固定的提示词模板,准确率下降 6.16% 至 85.31%。这表明政务查询具有高度的领域多样性,通过动态检索语义相似的少样本样例,能够为模型提供关键的上下文生成参考,增强系统对特定业务语境的自适应能力。尤其是 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 格式的样例比传统的 $\langle \text{Query}, \text{SQL} \rangle$ 格式能够提供更丰富的推理线索。

表 3-5 移除系统关键模块的消融实验性能比较

模型方法	执行准确率/%	ΔEX /%
Full Model	91.47	-
w/o HNSW	89.32	-2.15
w/o Dynamic Few-shot	85.31	-6.16
w/o LA-Schema	84.62	-6.85
w/o Logical Generator	87.93	-3.54
w/o DC Generator	88.88	-2.59
w/o Query Fixer	89.24	-2.23
w/o Selection Agent	86.12	-5.35

w/o LA-Schema. 当系统不使用逻辑增强模式而是采用传统的 DDL Schema 时, 性能出现最大的下滑, 降幅达 6.85%。值得注意的是, 该配置下的准确率 (84.62%) 与最强基线 XiYan-SQL (83.83%) 接近, 说明在去除 LA-Schema 后系统的性能优势几乎消失。这充分证明了在政务垂直领域, 物理字段名的隐晦性和业务逻辑的复杂性是 SQL 生成的核心瓶颈, 而 LA-Schema 通过显式注入业务等先验信息, 将复杂的逻辑演算简化为模式调用, 是系统高性能的核心保障。

w/o Logical Generator & w/o DC Generator. 分别去除逻辑映射生成器和渐进分治生成器后, 执行准确率分别下降 3.54% 和 2.59%。逻辑映射生成器的贡献略大于渐进分治生成器, 这反映了政务数据集中包含较多需要领域逻辑映射的业务指标查询。

w/o Query Fixer. 去除查询修复器后, 准确率下降 2.23% 至 89.24%。虽然修复器的单独贡献相对较小, 但其通过基于执行反馈的迭代修正机制, 有效提升了候选集中可执行查询的比例, 为后续判别模型提供了更高质量的输入。

w/o Selection Agent. 若不使用基于执行反馈的选择智能体进行候选判别, 而是采用一致性多数投票策略, 准确率下降 5.35% 至 86.12%。这说明多数投票无法充分感知 SQL 的执行质量, 而选择智能体能够通过执行反馈识别出语义虽不一致但逻辑正确的潜在答案, 显著提升了异构候选冲突时的容错率。

3.8.5 异构协同与候选判别分析

为深入剖析异构协同生成机制与候选判别模型这一核心创新的有效性, 从生成器独立性能、判别模型精度、候选池多样性三个维度设计了系统性的专项实验。

生成器性能分析

为揭示各生成器的独立贡献, 本文在候选判别前对每个生成器的单候选生成能力进行了评估。即将每个生成器生成单条候选 SQL 的准确率与零样本 CoT 基线

进行对比，同时考察查询修复器对各生成器的增益效果。结果如图3-5所示。

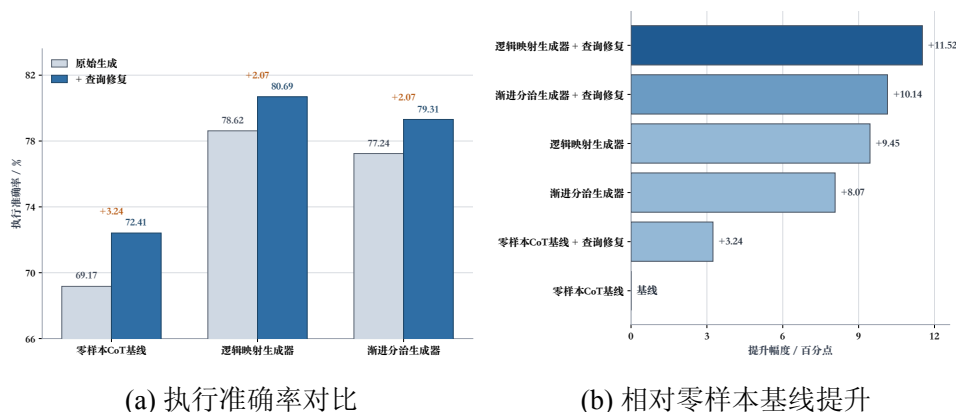


图 3-5 各生成器单候选生成性能对比

从结果中可以得出以下分析。首先，两个生成器的单候选性能均显著优于零样本 CoT 基线，逻辑映射生成器提升了 9.45 个百分点，渐进分治生成器提升了 8.07 个百分点，验证了本文所设计的生成策略在引导大语言模型进行 SQL 生成方面的有效性。其中逻辑映射生成器略优于渐进分治生成器，这主要归因于政务数据集中包含大量可直接利用 LA-Schema 预置逻辑块的业务指标查询。其次，查询修复器对所有生成方法均带来了约 2%~3% 的性能增益，表明基于执行反馈的迭代修复机制能够有效纠正语法错误和执行异常，提升候选的整体可用性。

判别模型分析

候选判别模型的二分类准确率直接决定了从候选池中选出正确答案的能力。本文对不同选择策略的判别精度进行了对比实验，在配对比较中仅统计一个候选正确、另一个候选错误的高价值情况。结果如表3-6所示。

表 3-6 不同选择策略的二分类判别准确率

选择模型	二分类准确率/%
Qwen3-Coder-30B（未微调）	59.38
Qwen3-8B（未微调）	55.62
Qwen3-8B（微调，本文方法）	72.85

实验结果表明，未经微调的大语言模型在配对判别任务上的准确率仅略高于随机猜测水平（55%~59%），这是因为异构生成器产生的候选 SQL 之间往往仅存在细微差异（如 WHERE 条件中的一个值不同、聚合范围略有差别等），未微调模型难以捕获这些关键区分特征。相比之下，经过在训练集上产生的高价值正负样本对进行微调后，Qwen3-8B 的判别准确率提升至 72.85%，提升了 17.23 个百分

点。这一结果与 CHASE-SQL^[6]中的发现一致，即配对选择任务需要通过监督微调使模型学习到候选 SQL 之间细粒度的语义偏好。

候选池多样性与生成器互补性分析

本文对两个生成器在候选池层面的互补性进行了深入考察。对于测试集中的每个问题，分别统计仅逻辑映射生成器产生正确候选、仅渐进分治生成器产生正确候选以及两者均产生正确候选的情况。如图3-6所示。

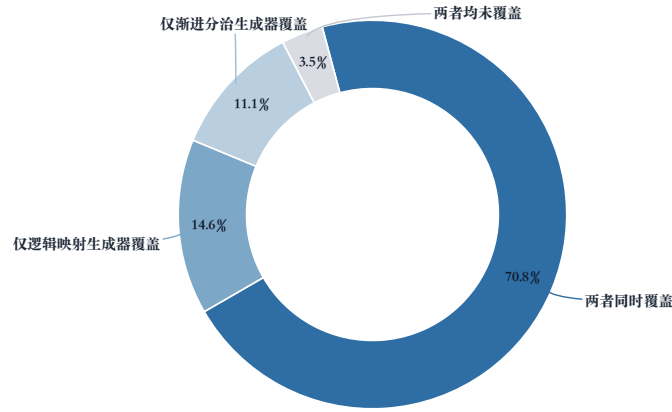


图 3-6 生成器在测试集覆盖统计

在 144 个测试样本中，有 21 个问题（14.6%）仅被逻辑映射生成器覆盖，16 个问题（11.1%）仅被渐进分治生成器覆盖，102 个问题（70.8%）被两个生成器同时覆盖，仅有 5 个问题（3.5%）未被任一生成器产生的候选所覆盖。

逻辑映射生成器独占的 21 个正确案例以包含预定义业务指标的标准化查询为主，涉及需要精确公式调用的问题。渐进分治生成器独占的 16 个正确案例则集中在结构复杂的查询上，包括多层嵌套子查询、跨表关联聚合等场景。两种推理范式在查询类型上的互补性是异构协同策略有效的根本原因。

3.8.6 效率分析

表3-7展示了本文方法各模块的平均耗时、时间占比、LLM 调用次数及 Token 消耗情况。

端到端单条查询存在波动，累计需 7~24 次 LLM 调用，Token 消耗范围为 12500~43000 个，具体取决于候选修复的轮次以及生成器生成的多样性。

其中候选生成环节占据了最大的时间比重（42.6%），平均耗时 17.1 秒，这主要是由于两个生成器各需采样多个候选并涉及较长的提示序列，尽管双路并行执

表 3-7 各模块效率分析（平均单样本）

模块	平均耗时/s	占比/%	平均 LLM 调用/次	Token 消耗/个
模式链接	7.8	19.5	3	3000–8000
候选生成（双路并行）	17.1	42.6	3	9000–20000
查询修复	3.8	9.5	0–3	0–6000
候选判别	10.7	26.7	1–15	500–9000
其他（SQL 执行等）	0.7	1.7	0	0
端到端总计	40.1	100.0	7–24	12500–43000

行已有效压缩了该环节的绝对耗时。每次候选生成固定调用 3 次 LLM（逻辑映射生成器 1 次、渐进分治生成器 2 次-均为线程数为 10 的并发调用），Token 消耗为 9000~20000 个，是所有模块中最高的。候选判别环节耗时 10.7 秒（26.7%），是第二大耗时模块。该环节需要对候选对执行多轮配对比较，LLM 调用次数为 1~15 次，波动较大的原因在于执行结果一致的候选对可直接跳过模型判别，而执行结果不同的候选对则需逐对调用选择智能体，Token 消耗为 500~9000 个。模式链接环节耗时 7.8 秒（19.5%），需 3 次 LLM 调用，其开销主要来自实体提取、向量检索及大语言模型的列相关性评估，Token 消耗为 3000~8000 个。查询修复环节耗时 3.8 秒（9.5%），LLM 调用次数为 0~3 次，Token 消耗为 0~6000 个。当所有候选 SQL 均执行成功时无需触发修复流程，调用次数和 Token 消耗均为零；当存在执行失败的候选时，最多进行 3 轮迭代修复。

对于政务数据查询这一非实时交互的应用场景而言，40 秒左右的响应时间处于可接受范围内。在实际部署中，候选生成的并行化策略（两个生成器同时运行且多线程生成候选 SQL）已将该环节的耗时控制在合理水平；同时 SQL 执行结果的缓存大大提高了使用性能。若需进一步降低延迟，可考虑减少每个生成器的采样数量（如降低至 5 或者 3），以在准确率与响应速度之间取得更优的折中。

3.8.7 公共数据集实验结果

BIRD 和 Spider 是两个广受认可的跨域数据集，分有不重叠的训练集、开发集、测试集。BIRD 包含来自 95 个大型数据库的超过 12,751 个独特的问题-SQL 对，涵盖 37 个专业领域，其数据库设计模仿现实世界场景，具有杂乱的数据行和复杂的模式。Spider 包含 10,181 个问题和 5,693 个独特的复杂 SQL 查询，涉及 200 个数据库，覆盖 138 个领域。

为评估本文方法在泛用场景下的适用性与稳健性，本文在两个主流的跨领域公共 Text-to-SQL 基准数据集上进行了实验：BIRD^[14]开发集（BIRD-dev）和 Spider^[15]测试集（Spider-test）。在公共数据集实验中，本文方法未使用政务领域知识

库，动态样例库也替换为基于目标数据集训练集构建的通用样例库，以确保评测条件与其他基线方法保持一致。各方法的执行准确率对比结果如表3-8所示。

表 3-8 公共数据集上的性能对比

方法	使用模型	BIRD/%	Spider/%
DeepSeek-V3	DeepSeek-V3	63.80	85.80
Qwen3-Coder	Qwen3-Coder-30B-A3B-Instruct	67.33	88.27
DAIL-SQL	GPT-4	54.76	86.60
OpenSearch-SQL	GPT-4o	69.30	87.10
Alpha-SQL	Qwen2.5-Coder-32B	69.70	-
XiYan-SQL	GPT-4o + Qwen2.5-Coder-32B	73.34	89.65
CHASE-SQL	Gemini-1.5-Pro + Gemini-1.5-Flash	74.90	87.60
OmniSQL	Qwen2.5-Coder-32B	67.00	89.80
本文方法	Qwen3-Coder-30B-A3B-Instruct + Qwen3-Flash-8B	71.23	89.25

从结果中可以得出以下分析：

(1) 在 **BIRD-dev** 上，本文方法取得了 71.23% 的执行准确率，仅次于 CHASE-SQL (74.90%) 和 XiYan-SQL (73.34%)。与政务数据集上的显著领先不同，本文方法在 BIRD 上未能超越这两个强基线，其核心原因在于：BIRD 数据集包含 95 个不同领域的数据库，本文的 LA-Schema 机制在缺乏目标领域知识库支撑的情况下无法发挥其领域逻辑增强的核心优势。而 CHASE-SQL 和 XiYan-SQL 针对通用场景设计了更具普适性的生成策略（如 CHASE-SQL 的查询执行计划 CoT 和 XiYan-SQL 的多生成器集成），在无领域先验的条件下表现更为稳健。尽管如此，本文方法仍显著优于 DAIL-SQL (+16.47%)、OmniSQL (+4.23%) 等方法，且相较于使用相同基础模型的 Qwen3-Coder 零样本基线提升了 3.90 个百分点，表明异构协同生成与微调判别机制本身在通用场景下同样具备增益能力。

(2) 在 **Spider-test** 上，本文方法取得了 89.25% 的执行准确率，仅次于 OmniSQL (89.80%) 和 XiYan-SQL (89.65%)，超过了 CHASE-SQL (87.60%)。Spider 数据集的查询结构相对规范、领域语义歧义较少，各方法的性能差距较 BIRD 上更为收窄。本文方法在未针对 Spider 做任何特定适配的情况下展现竞争力表现，说明本文设计的异构候选生成框架和配对判别机制具有较好的跨数据集泛化能力。值得注意的是，OmniSQL 通过百万级合成数据的大规模监督微调获得了最高性能，而本文方法仅对判别模型进行了小规模微调，在效率上更具优势。

(3) 综合来看，本文方法在公共数据集上表现出稳定的竞争力，验证了异构协同生成与微调判别这一技术框架的通用性。同时，本文方法在政务数据集上的领先优势 (91.47%) 与公共数据集上的性能差距也从侧面印证了 LA-Schema 领域逻

辑增强机制的核心价值，即在具备领域知识支撑的垂直场景中，本文方法能够充分发挥知识驱动的优势，实现显著超越；而在缺乏领域知识的通用场景中，方法仍能依靠异构生成与判别机制保持主流水平。

3.9 本章小结

本章针对政务数据查询中业务语义模糊、计算逻辑隐含及查询结构复杂等挑战，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法。该方法通过构建结构化领域知识库并以 LA-Schema 形式显式注入数据库模式，提升政务领域的业务理解能力；通过逻辑映射与渐进分治双路径异构生成机制，平衡了高频标准需求与低频复杂长尾需求；通过微调二分类判别模型与执行反馈修复机制，提升了最终查询的准确性与鲁棒性。最终在政务数据集和公开数据集验证了本章所提模型的有效性。为构建智能问数系统提供了核心技术支持。

第 4 章 面向政务洞察的分析脚本生成方法

4.1 引言

第三章围绕政务智能问数中的可靠取数问题，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法^[1]，重点解决自然语言查询到结构化查询语句的准确映射问题。然而，在真实政务业务场景中，用户需求往往并不止于“查到数据”，而是进一步希望系统能够围绕查询结果完成趋势识别、横向比较、结构刻画和结论归纳等分析任务。因此，仅有可靠的 SQL 生成能力仍不足以支撑完整的政务数据洞察闭环。

面向上述需求，本章研究政务智能问数场景下的 Text-to-Analysis 任务^[20-21]，即针对自然语言形式的分析需求，自动生成可执行的数据分析脚本，并输出图表、表格与简要文字洞察。与第三章关注“如何查准”不同，本章关注“如何分析准、展示稳”，其核心目标是在复用第三章可靠查询能力的基础上，构建从分析需求理解、分析计划生成、数据获取、脚本生成到受控执行与结果组织的完整链路。

本章将分析任务形式化定义为：给定自然语言分析需求 x 、第三章提供的查询生成管线 G_q 、分析算子库 $\mathcal{O}$ 以及脚本生成约束集合 $\mathcal{C}$ ，系统需要生成可执行分析脚本 s 及其最终结果包 R ，使得分析结果与用户意图最大化一致，即

$$(s^*, R^*) = \arg \max_{s \in \mathcal{S}, R \in \mathcal{R}} P(s, R \mid x, G_q, \mathcal{O}, \mathcal{C}) \quad (4-1)$$

其中， $\mathcal{S}$ 表示候选分析脚本集合， $\mathcal{R}$ 表示候选结果包集合。为支撑系统级闭环，本章将结果包统一组织为

$$\text{ResultPackage} = \{\text{charts}, \text{tables}, \text{insights}, \text{logs}\} \quad (4-2)$$

其中 **charts** 表示图表文件路径集合，**tables** 表示结构化表格结果，**insights** 表示文字洞察摘要，**logs** 表示执行状态与异常日志。

为控制任务边界并突出政务分析的典型应用场景，本文将目标任务限定为四类高频洞察类型：

(1) **趋势分析**：围绕时间维度识别指标变化轨迹，适用于财政收入变化、企业注册量波动、工单受理量走势等场景；

(2) **对比分析**：围绕区域、部门、年份等维度对关键指标进行横向比较，用于发现差距与优势；

(3) **排名分析**：围绕指定指标完成排序与头尾识别，适用于项目排名、预算排

名、效率排名等任务；

(4) **结构分析**：围绕整体内部组成关系计算占比与分布，用于分析财政支出结构、产业类型结构等问题。

与直接将自然语言需求端到端映射为长分析代码的做法^[22-23]相比，政务场景下的 Text-to-Analysis 还面临三类额外挑战。其一，用户分析需求通常隐含了指标、维度、时间范围和图表形式等多种约束，若缺乏中间规划过程，生成脚本极易偏离意图；其二，分析任务依赖的数据获取环节若与数据库直接耦合，则会重新暴露 Schema 理解和 SQL 正确性问题；其三，图表生成与脚本执行天然具有工程风险，若缺乏受控执行与统一输出机制，系统稳定性难以保障。

基于上述分析，本章以“规划驱动、约束生成、查询增强、闭环执行”为总体思路展开研究。具体而言：首先通过构建结构化中间表示 AP-Schema，将开放式分析需求转化为可执行的分析蓝图；其次利用轻量分析算子库与可视化规则，将分析过程约束为若干可组合的操作序列；再次将第三章查询生成管线作为统一数据获取接口接入分析流程，从源头保障输入数据可靠；最后通过受控执行与结果组织机制形成可观测、可回退的分析闭环。通过上述设计，第三章与第四章共同构成面向政务智能问数的完整技术链路，即第三章负责可靠取数，第四章负责自动分析与规范化展示。

4.2 总体框架设计

基于上述目标，本文设计了一个面向政务洞察的查询增强分析闭环框架，其整体流程由需求解析、AP-Schema 构建、分析算子编排、查询增强取数、约束脚本生成以及受控执行与结果组织六个阶段组成。与第三章的 Text-to-SQL 流程相比，本章并不重新解决数据库模式理解和 SQL 纠错问题，而是将第三章的方法作为标准化数据获取子模块嵌入分析流程中，实现“先查准、再分析”的层级协同。

框架的核心数据流可表示为：

$$\begin{aligned} A &= \text{Plan}(x), & Q_A &= \pi_q(A), & \mathcal{D} &= G_q(Q_A), \\ s &= \text{Generate}(A, \Gamma), & R &= \text{Execute}(s, \mathcal{D}) \end{aligned} \quad (4-3)$$

其中， A 表示由分析规划模块生成的 AP-Schema， $\pi_q(\cdot)$ 表示从 AP-Schema 中投影出取数需求子结构的操作， $\mathcal{D}$ 表示查询生成管线返回的数据结果与执行状态， Γ 表示字段元数据， s 表示最终分析脚本， R 表示结果包。

各阶段职责如下。

需求解析阶段负责从用户输入中识别分析对象、指标、时间范围、分组维度和

图示占位：面向政务洞察的查询增强分析闭环框架

自然语言分析需求 → 需求解析 → AP-Schema 构建 → 分析算子编排与可视化决策
→ 查询增强取数（调用第三章查询生成管线）→ 约束化脚本生成
→ 受控执行与结果组织 → ResultPackage

图 4-1 面向政务洞察的查询增强分析闭环框架

期望展示形式，为后续规划提供语义基础。

AP-Schema 构建阶段将自然语言需求转化为结构化分析表示，用于明确”分析什么、如何分析、如何展示”三个核心问题。该中间表示是本章的关键方法载体，也是后续脚本生成与结果验证的统一依据。

分析算子编排阶段根据任务类型与指标语义，从轻量分析算子库中选择并组织合适的操作链，如聚合、排序、占比、同比、透视等，同时依据可视化规则确定适配的图表类型，降低自由生成图表带来的不稳定性。

查询增强取数阶段不允许分析脚本直接拼接 SQL，而是将 AP-Schema 中的取数需求投影后交由第三章查询生成管线处理。这样既避免了重复解决复杂的数据库理解问题，也确保分析逻辑与取数逻辑的一致性。

约束脚本生成阶段采用”模板骨架 + 算子填充”的方式生成 Python 分析脚本^[22]。脚本在库依赖、输入接口、输出格式和异常处理方面均受到预定义约束，从而提升执行成功率与可维护性。

受控执行与结果组织阶段在沙箱环境中运行生成脚本，并将图表、表格、洞察文本和日志聚合为统一的 ResultPackage，以便上层系统展示和后续误差分析。

通过上述设计，本章的方法在保持大语言模型语义理解与代码生成能力的同时，引入了清晰的中间表示、稳定的算子编排机制和受控执行闭环，从而更适合政务场景下对可靠性与规范性的双重要求。

4.3 基于 AP-Schema 的规划驱动分析脚本生成

本节给出第四章的核心方法，即基于 AP-Schema 的规划驱动分析脚本生成方法。该方法并不将自然语言需求直接映射为完整代码，而是先生成结构化分析计划，再根据分析计划装配脚本，从而将高开放度的生成任务转化为受约束的规划与填充任务。该思路借鉴了思维链推理^[10]中”先规划后执行”的范式，相较于直接生成代码的方法，在分析步骤完整性、图表恰当性和执行稳定性方面更符合政务应用需求。

4.3.1 分析意图识别与 AP-Schema 构建

为统一表达分析需求，本文引入结构化中间表示 AP-Schema (Analysis Plan Schema)，并将其定义为

$$A = \langle T, M, D, F, O, V, S \rangle, \quad T \in \{\text{trend, comparison, ranking, structural}\} \quad (4-4)$$

其中， T 表示分析类型， M 表示指标集合， D 表示维度集合， F 表示筛选条件集合， O 表示分析算子序列， V 表示可视化规格， S 表示摘要要求。AP-Schema 既承担分析规划功能，也为后续的查询投影、脚本生成和结果验证提供统一接口。

表4-1给出了 AP-Schema 各字段的语义定义。

表 4-1 AP-Schema 字段定义

组成部分	符号	含义	典型内容
分析类型	T	描述任务属于趋势、对比、排名或结构分析中的哪一类	trend、ranking 等
指标集合	M	记录待分析指标及其聚合方式、单位、口径说明	收入金额/SUM、项目数量/COUNT
维度集合	D	记录用于分组、对比或展示的维度字段	区县、部门、年份、月份
筛选条件	F	记录时间范围、对象范围及其它过滤条件	年份 = 2024、区县 $\in \{A \text{ 区}, B \text{ 区}\}$
算子序列	O	记录分析过程中需要执行的操作链	group_agg $\rightarrow$ yoy $\rightarrow$ pivot
可视化规格	V	记录图表类型、坐标映射、标题和标注要求	折线图、横向柱状图、饼图
摘要要求	S	记录最终文字洞察需要覆盖的关键信息点	最高值、变化趋势、占比前三项

在具体生成过程中，系统首先对自然语言需求执行语义解析，识别其中显式或隐式包含的指标词、时间词、比较词、排序词和图表意图词，然后将这些信息映射到 AP-Schema 对应字段。例如，当用户提出“分析近两年各区一般公共预算收入的月度趋势，并给出同比变化”时，系统需要同时识别出指标“一般公共预算收入”、时间范围“近两年”、维度“区县”和“月份”、任务类型“趋势分析”，以及隐含的同比计算要求，从而形成完整的结构化表示。

AP-Schema 的引入带来两方面收益。其一，它将分析任务中的多种隐式约束显式化，减少了大语言模型在长代码生成过程中遗漏关键步骤的风险^[20]；其二，它将后续各模块的输入输出统一起来，使查询获取、脚本生成、执行修正和结果验证可以围绕同一中间表示协同工作。

上述过程可用算法4.1描述。

算法 4.1: 基于 AP-Schema 的分析规划算法

Input: 自然语言分析需求 x , 任务类型集合 $\mathcal{T}$, 分析算子库 $\mathcal{O}$, 可视化规则库 $\mathcal{V}$, 大语言模型 LLM

Output: 分析计划 $A = \langle T, M, D, F, O, V, S \rangle$

```

1 初始化:  $A \leftarrow \emptyset$ ;
2  $E \leftarrow \text{LLM}(x)$ ;           // 抽取指标、维度、时间范围、排序与图表意图
3  $T \leftarrow \text{IdentifyType}(E, \mathcal{T})$ ;           // 识别分析类型
4  $(M, D, F) \leftarrow \text{ParseSlots}(E)$ ;           // 填充指标、维度和筛选条件
5  $O \leftarrow \text{ArrangeOps}(T, M, D, F, \mathcal{O})$ ;           // 生成分析算子序列
6  $V \leftarrow \text{SelectVis}(T, D, O, \mathcal{V})$ ;           // 选择图表类型和展示字段
7  $S \leftarrow \text{BuildSummarySpec}(T, M, V)$ ;           // 生成摘要要求
8  $A \leftarrow \langle T, M, D, F, O, V, S \rangle$ ;
9 return  $A$ 

```

4.3.2 分析算子编排与可视化决策

为了避免分析脚本完全依赖自由代码生成, 本章进一步引入轻量分析算子库

$$\mathcal{O} = \{\text{group_agg}, \text{sort}, \text{topk}, \text{ratio}, \text{yoy}, \text{share}, \text{pivot}\} \quad (4-5)$$

并通过任务类型与语义规则对其进行编排。与第三章通过逻辑增强模式约束 SQL 生成的思想相类似, 本章通过算子库将常见政务分析逻辑拆解为有限、可组合、可验证的处理单元, 以降低分析脚本生成的不确定性。

针对四类典型任务, 本文设计如下算子映射关系:

$$\Psi(T) = \begin{cases} [\text{group_agg}, \text{yoy}/\text{pivot}], & T = \text{trend} \\ [\text{group_agg}, \text{pivot}, \text{sort}], & T = \text{comparison} \\ [\text{group_agg}, \text{sort}, \text{topk}], & T = \text{ranking} \\ [\text{group_agg}, \text{share}], & T = \text{structural} \end{cases} \quad (4-6)$$

其中, 趋势分析通常要求按时间粒度聚合, 并在需要时计算同比或环比; 对比分析需要完成分组聚合与维度展开; 排名分析需要在聚合基础上引入排序与截断; 结构分析则重点使用占比计算算子。

在算子定义层面, 若将聚合后的指标值记为 x_t , 则同比算子、占比算子和比率算子的典型计算形式可分别写为:

$$\text{YoY}_t = \frac{x_t - x_{t-p}}{x_{t-p}} \times 100\% \quad (4-7)$$

$$\text{Share}_i = \frac{x_i}{\sum_j x_j} \times 100\% \quad (4-8)$$

$$\text{Ratio}(x_a, x_b) = \frac{x_a}{x_b} \times 100\% \quad (4-9)$$

其中 p 表示同比的时间跨度，在年度任务中通常取 $p = 1$ ，在月度任务中通常取 $p = 12$ 。

基于算子序列，系统进一步执行可视化决策。考虑到图表类型与分析任务之间存在较强对应关系^[24-25]，本文不将图表选择完全交给自由生成，而是引入规则驱动的可视化选择函数：

$$v^* = \begin{cases} \text{line_chart}, & T = \text{trend} \\ \text{bar_chart}, & T \in \{\text{comparison}, \text{ranking}\} \\ \text{pie_chart}, & T = \text{structural} \text{ 且类别数} \leq 6 \\ \text{stacked_bar_chart}, & T = \text{structural} \text{ 且类别数} > 6 \text{ 或存在时间对比} \end{cases} \quad (4-10)$$

上述规则的主要考虑在于：趋势信息更适合使用折线图表达连续变化；排名与对比任务更适合使用柱状图突出数量差异；结构分析在类别数较少时可使用饼图，而在类别较多或存在时间维对比时，采用堆叠柱状图可避免扇区过多导致可读性下降。

通过“分析算子编排 + 可视化规则决策”的联合设计，系统将原本需要由模型自由完成的复杂分析逻辑转化为受约束的结构化操作链，从而显著降低遗漏关键处理步骤、使用不恰当图表类型以及输出风格不统一等问题。

4.3.3 约束化分析脚本生成

在获得 AP-Schema 后，脚本生成模块不再直接生成整段自由代码，而是采用“模板骨架 + 算子填充”的方式装配分析脚本。设基础模板、数据处理模板、算子模板、可视化模板和结果输出模板分别为 $\mathcal{T}_{base}$ 、 $\mathcal{T}_{data}$ 、 $\mathcal{T}_{op}$ 、 $\mathcal{T}_{vis}$ 和 $\mathcal{T}_{out}$ ，则脚本生成过程可表示为：

$$s = \mathcal{T}_{base} \oplus \mathcal{T}_{data}(A) \oplus \mathcal{T}_{op}(O) \oplus \mathcal{T}_{vis}(V) \oplus \mathcal{T}_{out}(S) \quad (4-11)$$

其中 $\oplus$ 表示模板片段按预定义顺序的装配操作。

该装配式生成遵循以下约束。

(1) **固定库白名单**:脚本仅允许使用 `pandas`^[26]、`numpy`、`matplotlib`^[27]和`seaborn`等白名单库，以避免导入不稳定或不安全依赖。

(2) **统一输入接口**:脚本的数据输入不是数据库连接，而是第三章查询生成管线返回的 `DataFrame` 对象及其字段元数据，从而实现数据获取与分析处理的解耦。

(3) **标准化代码骨架**:所有脚本都包含数据检查、算子执行、图表绘制、摘要生成和结果打包五个逻辑区块，减少生成风格漂移。

(4) **统一输出契约**:脚本最终必须输出符合式(4-2)定义的 `ResultPackage`，便于执行器统一接收和上层系统展示。

(5) **静态安全检查**:对于 `eval`、`exec`、`subprocess`、`socket` 等高风险调用，脚本生成后需执行静态规则检查，不满足约束的候选将被直接拒绝。

在约束化生成机制下，模型只需要为模板中的若干槽位填入字段名、算子参数和图表配置，而不必从零构造整个分析程序。下述简化骨架展示了该方法的基本结构：

```
df = load_query_result()
df = apply_operators(df, 0)
fig = render_chart(df, V)
insights = summarize(df, S)
package = build_result(fig, df, insights, logs)
```

算法4.2给出了约束化分析脚本生成的整体过程。

应当指出，约束化脚本生成并不否定大语言模型的作用，而是将其生成能力置于更稳定的工程边界内。一方面，模型仍负责理解任务语义、补全算子参数和摘要逻辑；另一方面，关键的输入输出协议、可调用库范围和输出格式则由系统侧强约束。该设计使得本章方法在学术上体现为“规划驱动+模板约束”的方法创新，在工程上则体现为更高的执行稳定性与可维护性。

4.4 基于查询增强的分析执行闭环

仅有分析规划与脚本装配仍不足以支撑稳定的政务分析系统。为确保生成的脚本能够在真实场景中可靠运行，本节进一步给出第四章的执行闭环设计，重点回答两个问题：其一，第四章如何在数据获取阶段与第三章形成明确而稳定的接口；其二，生成脚本如何在受控环境中执行并形成统一结果回传。

算法 4.2: 脚本装配与受限生成算法

Input: AP-Schema $A = \langle T, M, D, F, O, V, S \rangle$, 字段元数据 Γ , 模板集合 $\mathcal{T}$, 白名单库 $\mathcal{L}_w$, 禁用接口集合 $\mathcal{B}$, 大语言模型 LLM

Output: 分析脚本 s

```

1 初始化:  $s \leftarrow \mathcal{T}_{base}$ ;
2  $s \leftarrow s \oplus \text{BuildImport}(\mathcal{L}_w)$ ; // 插入固定导入库
3  $s \leftarrow s \oplus \text{BindInput}(\Gamma)$ ; // 绑定查询结果输入接口
4 foreach  $o_i \in O$  do
5    $s \leftarrow s \oplus \text{FillOpTemplate}(o_i, A, \Gamma, \text{LLM})$ ; // 填充算子模板
6 end
7  $s \leftarrow s \oplus \text{FillVisTemplate}(V, A, \Gamma)$ ; // 生成绘图代码
8  $s \leftarrow s \oplus \text{FillSummaryTemplate}(S, A)$ ; // 生成摘要与结果打包代码
9 if  $\text{StaticCheck}(s, \mathcal{B}) = \text{False}$  then
10    $s \leftarrow \text{RejectAndRegenerate}(A, \Gamma)$ ; // 拒绝高风险脚本并重新生成
11 end
12 return  $s$ 

```

4.4.1 基于查询增强的数据获取

本章方法的一个关键原则是：分析脚本不直接面向数据库，而是统一通过第三章的查询生成管线获取数据。为此，本文从 AP-Schema 中定义取数需求投影函数 $\pi_q(\cdot)$ ，将分析计划中与数据获取相关的部分映射为查询意图：

$$Q_A = \pi_q(A) = \langle M, D, F \rangle \quad (4-12)$$

随后，第三章查询生成管线根据 Q_A 生成并修复 SQL，最终返回数据契约：

$$\mathcal{D} = G_q(Q_A) = \langle \mathbf{X}, \Gamma, \sigma \rangle \quad (4-13)$$

其中， $\mathbf{X}$ 表示以 DataFrame 形式封装的查询结果， Γ 表示字段名称、数据类型、单位和行数等元数据， σ 表示查询执行状态。

这一设计使得第三章与第四章在职责上形成清晰分工：第三章解决“如何从复杂政务数据库中正确取数”，第四章解决“如何对已获取的数据进行分析与展示”。二者虽在功能上紧密协同，但并不在方法上相互侵入，因此能够有效避免在分析阶段重复暴露 Schema 歧义、字段映射错误和 SQL 逻辑幻觉等问题。

查询增强机制带来三点直接收益。第一，它将查询正确性风险前移至第三章已验证过的模块中，避免分析脚本重复承担数据库理解任务。第二，返回的字段元数据为第四章的可视化决策和结果校验提供了辅助依据，例如单位信息可以用于纵轴标签设置，字段类型可以用于日期转换与聚合方式判断。第三，查询状态 σ 为后续执行闭环提供了异常边界，若数据获取失败，则可在分析脚本生成前终止

图示占位：查询管线与分析管线协同示意图

AP-Schema 中的指标/维度/筛选条件子结构 → 查询意图投影 $\pi_q(A)$
 → 第三章查询生成管线（Schema 理解、SQL 生成、执行修复）
 → 返回 DataFrame + 字段元数据 + 执行状态
 → 约束化分析脚本处理与结果生成

图 4-2 查询管线与分析管线协同示意图

流程，避免无效执行。

4.4.2 受控执行与结果组织

在获得查询结果后，系统需要保证分析脚本能够在可控条件下稳定执行。为此，本文设计了轻量级受控执行机制^[23]，在不展开系统实现细节的前提下，将其抽象为包含脚本执行、异常捕获、有限修正与结果聚合的统一闭环。

首先，所有脚本均在受限执行环境中运行，执行环境仅开放白名单库，并限制文件系统写入范围、网络访问权限和系统调用能力。同时为单次任务设置统一的运行上限，本文实验中将超时阈值设置为 300 秒，并对 CPU 和内存使用进行监控，以避免低质量脚本占用过多资源。

其次，对于脚本执行过程中出现的异常，系统记录标准输出、标准错误和异常栈信息，并将其写入日志字段中。当错误属于字段缺失、类型不匹配或绘图参数不一致等可修复类别时，系统允许进行有限次修正^[12]。设第 k 轮执行脚本为 s_k ，对应异常为 e_k ，则执行闭环可表示为：

$$R_k = \begin{cases} \text{Execute}(s_k, \mathbf{X}), & \text{脚本执行成功} \\ \text{Execute}(\text{Repair}(s_k, e_k), \mathbf{X}), & \text{脚本执行失败且 } k < \beta \\ \text{FailPackage}(e_k), & \text{其他情况} \end{cases} \quad (4-14)$$

其中 β 表示最大修正次数。考虑到分析脚本较长且修复代价较高，本文取 $\beta = 2$ ，即在初次执行失败后最多进行两轮有限修正。

最后，脚本执行成功后，系统将图表路径、表格结果、摘要文本和日志统一聚合为 **ResultPackage** 返回给上层应用。这一统一输出契约的意义在于：其一，前端展示层不再需要解析不同脚本的异构输出；其二，后续实验评估可以直接围绕图表、表格与摘要是否完整展开；其三，失败任务也可以通过日志字段保留诊断信息，便于误差分析。

通过“查询增强 + 受控执行 + 统一结果组织”的联合设计，第四章的方法不仅能够生成分析脚本，而且能够在工程上形成一个可观测、可修正、可回退的稳定

闭环。这也是本章区别于一般 Text-to-Code 方案的重要特征。

4.5 实验与结果分析

本节旨在验证本章方法的两项核心主张：其一，规划驱动的 AP-Schema 与分析算子编排是否能够提升分析脚本生成的稳定性与任务完成率；其二，复用第三章查询生成管线是否能够显著提升数据获取正确性并最终改善分析结果质量。与第三章面向 Text-to-SQL 的大规模基准评测不同，本章实验强调中等规模的任务覆盖和端到端闭环验证。

4.5.1 测试任务与实验设置

实验任务基于第三章构建的同源政务数据库场景展开，仍使用财政预算、公共资源和政务项目等领域的 5 个数据库、33 张数据表作为底层数据来源。为了贴近本章的分析定位，本文在此基础上重新构建了 28 个自然语言分析任务，其中趋势分析 8 个、对比分析 7 个、排名分析 7 个、结构分析 6 个。每个任务均配备人工核验的参考分析目标，包括目标指标、期望图表类型和关键结论要点。

为保持与第三章实验环境的一致性，本章在分析规划与脚本生成阶段仍采用 Qwen3-Coder-30B-A3B-Instruct^[16]作为基础模型，数据获取环节直接复用第三章的完整查询生成管线。脚本执行环境基于 Python 3.9, 预装 pandas 1.5.3^[26]、matplotlib 3.7.1^[27]、seaborn 0.12.2 和 numpy 等依赖，单任务超时阈值为 300 秒。对于执行失败脚本，最多进行两轮有限修正。

本文采用以下四个指标对结果进行评估。

代码执行成功率 (Execution Success Rate, ESR)，表示生成脚本能够完整执行结束的比例：

$$ESR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{exec}_i = 1] \quad (4-15)$$

分析任务完成率 (Task Completion Rate, TCR)，表示不仅脚本执行成功，而且输出图表、表格和摘要在逻辑上满足原始分析意图的比例：

$$TCR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{task}_i = 1] \quad (4-16)$$

结果正确性 (Result Accuracy, RA)，表示在任务完成样本中，系统输出与人

工参考结果一致的比例。对于数值结果，若相对误差不超过 $\epsilon = 3\%$ 则视为正确：

$$RA = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbb{I}[\delta(y_i^{pred}, y_i^{ref}) \leq \epsilon] \quad (4-17)$$

其中 N_c 表示任务完成的样本数， $\delta(\cdot)$ 表示相对误差或人工一致性判别函数。

图表恰当性 (Chart Appropriateness, CA)，由 3 名标注者从图表类型选择、坐标映射、标题与标签完整性三个维度进行 1 到 5 分评分，取平均值：

$$CA = \frac{1}{3N_c} \sum_{i=1}^{N_c} \sum_{j=1}^3 \text{score}_{ij} \quad (4-18)$$

为验证两项核心设计，本文设置如下三组方法进行比较：

(1) **Direct-Code**: 直接将自然语言分析需求端到端映射为 Python 脚本^[22]，不经过 AP-Schema 规划，也不使用算子编排机制；

(2) **Plan+In-script-SQL**: 采用 AP-Schema 和约束化脚本生成，但在脚本内部自由拼接并执行 SQL，而不是调用第三章查询生成管线；

(3) **Ours**: 本文完整方法，即“AP-Schema 规划 + 算子编排 + 查询增强取数 + 约束化生成 + 受控执行”。

4.5.2 结果对比与消融分析

表4-2给出了三种方法在 28 个政务分析任务上的总体结果。可以看出，本文方法在执行稳定性、任务完成率、结果正确性和图表恰当性四个指标上均明显优于两个对比方法。

表 4-2 不同分析脚本生成方法的总体实验结果

方法	ESR/%	TCR/%	RA/%	CA/分
Direct-Code	57.1	39.3	72.7	3.2
Plan+In-script-SQL	75.0	57.1	81.3	3.9
Ours	92.9	85.7	95.8	4.6

从结果可以得到如下结论。

第一，规划驱动机制显著提升了脚本稳定性。与 Direct-Code 相比，Plan+In-script-SQL 在 ESR 上提升了 17.9 个百分点，在 TCR 上提升了 17.8 个百分点。这表明 AP-Schema 与分析算子编排有效降低了脚本生成的随意性，使得模型更少遗漏关键处理步骤，如时间字段转换、分组聚合顺序和图表轴映射等。

第二，复用第三章查询生成管线是提升整体性能的关键。在已经采用规划驱

动的前提下，Ours 相较于 Plan+In-script-SQL 在 TCR 上进一步提升 28.6 个百分点，在 RA 上提升 14.5 个百分点。这说明在脚本内部自由生成 SQL 仍然会重新暴露字段误解、连接错误和过滤条件偏差等问题，而第三章查询生成管线能够有效提供更可靠的数据输入，使第四章方法将注意力集中在分析本身。

第三，图表恰当性受益于可视化规则约束。 Ours 在 CA 上达到 4.6 分，说明通过任务类型到图表类型的规则映射，系统生成的图表在类型选择、标题描述和坐标配置上更加规范，较少出现趋势任务使用饼图、结构任务使用不合理折线图 etc 不匹配现象。

为了进一步直接验证两项核心主张，表4-3给出了按主张组织的关键对比结果。

表 4-3 关键设计主张的对比与消融结果

对比项	较弱配置 (TCR/RA)	较强配置 (TCR/RA)	提升
规划驱动 vs. 直接生成 +8.6 RA	直接生成 (39.3 / 72.7)	规划 + 脚本内 SQL (57.1 / 81.3)	+17.8 TCR
查询增强 vs. 脚本内自由 SQL +14.5 RA	规划 + 脚本内 SQL (57.1 / 81.3)	本文方法 (85.7 / 95.8)	+28.6 TCR

结合错误案例可以发现，Direct-Code 的失败主要表现为三类：一是省略必要的数据处理步骤，例如仅完成取数却未进行同比、占比或排序；二是生成不可执行代码，例如误用不存在的字段或绘图库接口；三是图表表达与任务意图不匹配。相比之下，Plan+In-script-SQL 虽然通过规划约束降低了部分执行错误，但其失败仍大量集中在取数阶段，例如错误地理解字段口径、拼接错误的筛选条件或遗漏必要的分组字段。本文完整方法通过复用第三章查询生成管线，从源头避免了这些问题，因此能在整体闭环上获得显著优势。

4.5.3 案例分析

为更直观展示本章方法的端到端工作过程，本节选取两个具有代表性的案例进行分析，分别对应趋势分析和结构分析场景。

案例一：各区一般公共预算收入年度趋势分析

用户需求：分析 2023 年至 2024 年各区一般公共预算收入的月度变化趋势，并给出同比情况。

AP-Schema 生成结果：根据需求解析与规划模块，系统生成的分析计划可表

示为

$$A_1 = \langle \text{trend}, \{ \text{一般公共预算收入/SUM} \}, \{ \text{区县, 月份} \}, \\ \{ \text{年份} = 2023-2024 \}, [\text{group_agg}, \text{yoy}, \text{pivot}], \\ \text{line_chart}, \text{趋势与同比摘要} \rangle \quad (4-19)$$

查询增强取数：系统将 A_1 中的 $\langle M, D, F \rangle$ 投影为查询意图，并调用第三章查询生成管线返回包含 `district_name`、`month` 和 `revenue` 三列的结果数据。

关键脚本片段：在脚本生成阶段，算子序列被装配为如下核心处理逻辑：

```
agg = df.groupby(['district_name', 'month'])
      ['revenue'].sum().reset_index()
agg['yoy'] = agg.groupby('district_name')
              ['revenue'].pct_change(12) * 100
```

图示占位：案例一输出折线图

展示 2023 年至 2024 年各区一般公共预算收入月度折线趋势，
并在关键月份标注同比变化幅度

图 4-3 案例一趋势分析结果示意图

结果洞察：系统输出的折线图如图4-3所示。对应摘要指出：2024 年各区一般公共预算收入总体延续增长趋势，其中 A 区在第二季度增速最为明显，5 月至 7 月连续高于上年同期；B 区波动较小但整体保持稳健增长。同比结果显示，A 区在 2024 年 6 月达到最高同比增幅，说明该区在年中阶段财政收入扩张较快。

案例二：财政支出结构分析

用户需求：展示 2024 年财政支出在不同职能领域上的构成比例，并指出占比最高的三类领域。

AP-Schema 生成结果：系统生成的分析计划为

$$A_2 = \langle \text{structural}, \{ \text{支出金额/SUM} \}, \{ \text{支出领域} \}, \\ \{ \text{年份} = 2024 \}, [\text{group_agg}, \text{share}, \text{sort}], \\ \text{pie_chart}, \text{占比与前三项摘要} \rangle \quad (4-20)$$

查询增强取数：系统调用第三章查询生成管线获取 2024 年不同支出领域对应的财政支出金额，并返回 `function_type` 和 `amount` 等字段。

关键脚本片段：结构分析对应的核心处理逻辑为

```
agg = df.groupby('function_type')
      ['amount'].sum().reset_index()
agg['share'] = agg['amount'] /
      agg['amount'].sum() * 100
agg = agg.sort_values('share', ascending=False)
```

图示占位：案例二输出结构图

展示 2024 年财政支出在教育、城建、社会保障、医疗等领域的占比，
并突出占比前三项

图 4-4 案例二结构分析结果示意图

结果洞察：系统生成的结构图如图4-4所示。结果表明，教育、城市建设和社会保障三类支出合计占比超过全部财政支出的六成，其中教育支出占比最高，说明公共服务与民生保障仍然是当年财政投入重点。该案例同时验证了结构分析任务中占比计算和图表类型选择规则的有效性。

上述两个案例表明，本文方法能够将自然语言需求稳定转化为“AP-Schema 规划 - 查询增强取数 - 约束脚本生成 - 受控执行 - 洞察输出”的完整链路。相较于只给出查询结果的系统，本章方法进一步提升了数据的可解释性和可展示性，也使第三章与第四章在技术上形成了完整闭环。

4.6 本章小结

本章围绕政务智能问数中的 Text-to-Analysis 环节，提出了面向政务洞察的分析脚本生成方法。该方法以第三章的可靠查询能力为基础，引入 AP-Schema 作为结构化中间表示，将开放式分析需求转化为可执行的分析蓝图；通过轻量分析算子库与可视化规则，将常见政务分析逻辑组织为稳定的操作链；通过约束化脚本生成和受控执行机制，实现分析脚本的规范生成、稳定执行与统一结果组织；通过查询增强取数设计，使第三章与第四章在职责上清晰分工、在功能上形成闭环。

实验结果表明，本文方法在代码执行成功率、分析任务完成率、结果正确性和图表恰当性等指标上均显著优于直接生成和脚本内自由 SQL 两类基线方法，验证了“规划驱动优于直接生成”和“查询增强优于脚本内自由取数”两项核心主张。由此，第三章解决了问数场景中的可靠取数问题，第四章进一步解决了基于查询结果的自动分析与可视化表达问题，二者共同构成了面向政务领域的大模型数据查询与分析技术链路。

第 5 章 系统设计与实现

5.1 引言

5.2 系统总体架构设计

5.3 业务流程设计

5.3.1 智能查询业务流程

5.3.2 智能分析业务流程

5.3.3 异常回退与结果追溯流程

5.4 系统数据存储

5.4.1 业务数据源接入与元数据抽取

5.4.2 系统数据库设计

5.4.3 结果、日志与缓存管理

5.5 多智能体协同与上下文管理实现

5.5.1 轻量级多智能体角色设计

5.5.2 查询与分析服务编排

5.5.3 上下文维护与状态流转

5.6 系统功能实现与效果展示

5.7 本章小结

结论与展望

致 谢

“诶实扬华，自强不息。”时光荏苒，在交大度过的七载春秋，不仅是我求学生涯中最宝贵的时光，更是我人生观与价值观成型的关键阶段。回首往昔，心中满是感激与敬畏。

首先感谢我的恩师郑宇教授。郑老师不仅在科研上给予我启发式的指引，更在工作态度与人生认知上教会了我深刻的方法论。郑老师治学严谨、精益求精的学者风范，以及对前沿技术的敏锐洞察，令我受益终生。感恩郑老师在我迷茫时的悉心点拨，这份师恩，我将永远铭记于心，并在未来的职业道路上以此自勉。

感谢何华均师兄和麻志鹏师兄。你们严谨认真的科研精神和青云直上的实践态度，潜移默化地影响着我。感谢你们在我遇到瓶颈时提供的无私指导，让我在科研的道路上走得更加踏实。

研途漫漫，感谢在京东科技度过的充实时光。这里几乎承载了我研究生生涯的大半记忆。特别感谢韩博洋和孟垂实两位老师的悉心指导与包容，让我在工业界实战中飞速成长。感谢张晓龙、王璐璐、赵大燕、黎世骄、陈海乐等师兄师姐的并肩作战与温情陪伴，你们极大地丰富了我的生活色彩。同时，也感谢师弟虞杰杰、巴聪聪在工作与科研中的默契协作。

感恩在清华大学智能产业研究院以及字节跳动的宝贵经历。让我深刻体会到了“敢为极致、勇攀高峰”的精神。这些经历磨炼了我的韧性，也拓宽了我看世界的视野。

特别感谢陪伴我走过七年岁月的同门挚友：徐梓航、崔智源、柴江波、温海泉。我们的情谊始于拔尖班的初见，终于求职毕业的相守。科学研究表明，人体细胞每七年就会完成一次整体的更新；何其有幸，在这七年里，是你们陪伴我见证我从一个自我向另一个崭新自我的蜕变。其深厚羁绊，是我求学路上最温情的收获。

最后，感谢那个曾在多地奔波漂泊、却从未放弃梦想的自己。感谢自己在无数个深夜的坚持，在面对未知与挑战时的勇气。愿此去繁花似锦，归来仍是少年。

参考文献

- [1] Liu J, Lin J, Huang S, et al. A survey of text-to-SQL: advances, challenges, and future directions [J]. The VLDB Journal, 2025, 34(1): 1–35.
- [2] Chen J, Xiao S, Zhang P, et al. BGE M3-embedding: multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation [J]. arXiv preprint arXiv:240203216, 2024.
- [3] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 42(4): 824–836.
- [4] Gao D, Wang H, Li Y, et al. Text-to-SQL empowered by large language models: a benchmark evaluation [C]. Proceedings of the VLDB Endowment, 17(5), 2024: 1132–1145.
- [5] Xie X, Zhang G, Wang F, et al. OpenSearch-SQL: enhancing text-to-SQL with dynamic few-shot and consistency alignment [J]. arXiv preprint arXiv:250214913, 2025.
- [6] Pourreza M, Rafiei D. CHASE-SQL: multi-path reasoning and preference optimized candidate selection in text-to-SQL [J]. arXiv preprint arXiv:241001943, 2024.
- [7] Gao Y, Wang Y, Li Y, et al. XiYan-SQL: a multi-generator ensemble framework for text-to-SQL [J]. arXiv preprint arXiv:241108599, 2024.
- [8] TBD. Placeholder reference [J]. TBD, 2024.
- [9] TBD. Placeholder reference [J]. TBD, 2024.
- [10] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [C]. Advances in neural information processing systems, 35, 2022: 24824–24837.
- [11] Talaei S, Pourreza M, Chang Y C, et al. CHESS: contextual harnessing for efficient SQL synthesis [J]. arXiv preprint arXiv:240516755, 2024.
- [12] Shinn N, Cassano F, Gopinath A, et al. Reflexion: language agents with verbal reinforcement learning [C]. Advances in neural information processing systems, 36, 2024: 8634–8652.
- [13] TBD. Placeholder reference for isft [J]. TBD, 2024.
- [14] Liu J, Li D, Chen B, et al. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs [J]. Advances in neural information processing systems, 2025, 37: 61588–61617.

- [15] Yu T, Zhang R, Yang K, et al. Spider: a large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task [C]. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018: 3911–3921.
- [16] Bai J, Bai S, Chu Y, et al. Qwen technical report [J]. arXiv preprint arXiv:230916609, 2024.
- [17] DeepSeek-AI. DeepSeek-V3 technical report [J]. arXiv preprint arXiv:241219437, 2024.
- [18] Li B, Mou Y, Li J, et al. Alpha-SQL: zero-shot text-to-SQL via monte carlo tree search [J]. arXiv preprint arXiv:250217600, 2025.
- [19] Li J, Hui B, others. OmniSQL: synthesizing high-quality text-to-SQL data at scale [J]. arXiv preprint arXiv:250302164, 2025.
- [20] Hong S, Lin Y, Liu B, et al. Data interpreter: an LLM agent for data science [J]. arXiv preprint arXiv:240218679, 2024.
- [21] Ma P, Ding R, Wang S, et al. InsightPilot: an LLM-empowered automated data exploration system [C]. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2023: 346–360.
- [22] Chen M, Tworek J, Jun H, et al. Evaluating large language models trained on code [J]. arXiv preprint arXiv:210703374, 2021.
- [23] Qin B, Chen S, Zhu Y, et al. TaskWeaver: a code-first agent framework [C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2024: 262–273.
- [24] Dibia V. LIDA: a tool for automatic generation of grammar-agnostic visualizations and infographics using large language models [J]. arXiv preprint arXiv:230302927, 2023.
- [25] Maddigan P, Susnjak T. Chat2VIS: generating data visualisations via natural language using ChatGPT, Codex and GPT-4 [C]. IEEE Access, 11, IEEE, 2023: 45181–45193.
- [26] McKinney W. pandas: a foundational Python library for data analysis and statistics [C]. Python for High Performance and Scientific Computing, 14(9), 2011: 1–12.
- [27] Hunter J D. Matplotlib: a 2D graphics environment [J]. Computing in Science & Engineering, 2007, 9(3): 90–95.

附录 A

攻读硕士学位期间取得的研究成果

- 一、攻读硕士学位期间发表的论文及科研成果
- 二、参与科研项目/科研获奖等